Contents

BRICKS Application Programming Refrence

Document	Bricks Application Programming Reference
Edition	First Edition
Applies to	Bricks Transaction Server, version 1.5 and later
Companion	README.md — installation, configuration, and operations

---

About this document

This reference describes the application programming interface of the Bricks Transaction Server: the `EXEC CICS` command set, teh REXX and COBOL dialects that may issue those commands, the BMS-flavoured map DSL used to build 3270 panels, and the catalogue of sample programs shipped under `runtime/`.

Operational and installation topics — running `bricks`, editing `bricks.cnf`, signing on through CSSN, the CEMT master terminal, the `bricksload` stress tester, and the `/metrics` endpoint — are documented in the companion README.md.

Who should read this manual

This manual is for application programmmers who write transactions that run under Bricks. Familiarity with the IBM CICS programming model (pseudo-conversational dispatch, EIB, COMMAREA, BMS maps, KSDS files, temporary storage queues) is assumed; where bricks deviates from the IBM behaviour the difference is documnted explicitly.

Conventions used

- **`UPPERCASE`** in syntax — keywords; coded literally.
- **`lowercase`** in syntax — supplied by the programmer (a name or expression).
- **`[ ]`** — optional clauses.
- **`{ a | b }`** — choose one of teh alternatives.
- **`...`** — the preceding clause may repeat.

- Command syntax is shown in `EXEC CICS … END-EXEC` form. The bare-string form (`"VERB OPTIONS"` under `ADDRESS CICS`, REXX only) is described in Chapter 2; both forms dispatch identically.

Notation in this manual

Each `EXEC CICS` command in Part 2 is documented with the same five sections, in this order:

1. **Format** — the command syntax as a code block.
2. **Description** — what the command does and any important runtime behaviour.
3. **Options** — every keyword that follows the verb, with its meaning and any value constraints.
4. **Conditions** — the `EIBRESP` values the command may return, together with the cause of each.
5. **Example** — a short, runnable code fragment illustrating typical use.

Contents

Part 1. The bricks programming model

Chapter 1. Overview

A bricks **transaction** is a 4-character TRANSID listed in `runtime/transactions.conf`. Each transaction names a **program** — either a REXX source file in `runtime/rexx/` or a COBOL source file in `runtime/cobol/` — and an optional access-control list:

```
HELO:rexx:hello.rexx
HELP:rexx:help.rexx:public
QAGE:rexx:qage.rexx:public,users,admin
HELC:cobol:hello.cob:public
GUST:cobol:gust.cob:public
```

The first time a TRANSID is dispatched, its program is parsed and the AST is cached (see *Parsed-program cache* in the README); subsequent dispatches skip teh parse. Each running task has its own heap and stack — there is no shared mutable state between concurrently running tasks of the same TRANSID.

Programs interact with three layers:

1. **The 3270 terminal**, through `EXEC CICS SEND MAP` / `RECEIVE MAP` (and the unstructured `SEND TEXT` / `RECEIVE` pairing). Maps are defined in the BMS-style DSL described in Chapter 3.
2. **Persistent data**, through KSDS file commands (Chapter 7) and TS-queue commands (Chapter 9). Both are backed by an embedded bbolt B+tree at `data/files.boltdb`.
3. **Other programs**, through `LINK` (synchronous, with COMMAREA), `XCTL` (transfer of control, no return), and `RETURN TRANSID` (pseudo-conversational chaining). The Execute Interface Block (EIB) exposes session and request state (Chapter 11).

REXX and COBOL share **the same `EXEC CICS` dispatcher** (`cics/handler.go`). Every command documented in Part 2 is therefore available, with identical syntax and identical semantics, from either language.

A typical pseudo-conversational task has the shape:

```
ADDRESS CICS

EXEC CICS RECEIVE MAP('CUST1')              END-EXEC   /* read prior screen */
... validate / compute ...
EXEC CICS SEND    MAP('CUST1') FROM(SCR.) ERASE END-EXEC
EXEC CICS RETURN  TRANSID('CUST')           END-EXEC   /* chain back to self */
```

The dispatcher invokes the program once per ENTER. State that must survive between invocations rides in the COMMAREA on `RETURN`; the chained task receives it through `EIBCALEN` and the `DFHCOMMAREA` data area.

Chapter 2. The EXEC CICS command environment

Surface forms

Bricks accepts two equivalent surface forms inside an `ADDRESS CICS` scope.

1. IBM-canonical `EXEC CICS … END-EXEC` — the form CICS programmers recognize. Available from REXX and COBOL.

```
ADDRESS CICS

EXEC CICS ASSIGN  USERID(USR) TERMID(TRM)       END-EXEC
EXEC CICS SEND    MAP('HELO1') FROM(SCR.) ERASE  END-EXEC
EXEC CICS RETURN                                 END-EXEC
```

In REXX, a small preprocessor (`rexx/preprocess.go`, called from `rexx.Parse`) rewrites each `EXEC CICS … END-EXEC` block into the equivalent quoted-string command before lexing. Multi-line bodies are collapsed into a single command string; trailing newlines preserve source line numbers in error messages. Comments and string literals are left alone.

In COBOL, the parser collects every token between `EXEC CICS` and `END-EXEC` and reconstructs the body verbatim for the same `cics.ParseCommand` REXX uses.

2. Bare string under `ADDRESS CICS` — terser; available **only in REXX**.

```
ADDRESS CICS
"ASSIGN USERID(USR) TERMID(TRM)"
"SEND MAP('HELO1') FROM(SCR.) ERASE"
"RETURN"
```

Both forms route through `cics.ParseCommand` and dispatch to the same handler. After every command the handler writes `EIBRESP`, `EIBRESP2`, and `RC` into the program's frame.

Argument-passing rules

Inside the parentheses of an option:

- A bare identifier is treated as a **variable name**. The handler reads the value at runtime, and (where the option is an output) writes the result back into that variable.
- A quoted string literal (`'...'` or `"..."`) is treated as a **literal value**. The handler does not write to literals; passing a literal to an output-only option is rejected with `INVREQ`.

This distinction matters most for length and key fields: `LENGTH(LEN)` both reads the requested length on input and writes the actual length back; `LENGTH(80)` only sets the input length.

Programming model summary

Concept	Bricks implementation
Task	One invocation of one TRANSID (`session.TxCB`).
Program	A parsed REXX or COBOL source file, cached at L1/L2.
Pseudo-conversational chain	`EXEC CICS RETURN TRANSID(t) COMMAREA(d)`.
Conversational	Loop in the program; each `RECEIVE MAP` blocks on input.
Inter-program call	`LINK PROGRAM(name) COMMAREA(var)` (synchronous).
Storage shared across tasks	KSDS files, TS queues.
Storage shared within one task	REXX variables / COBOL WORKING-STORAGE only.
Implicit task end	Program runs off the end, or `STOP RUN` (COBOL).
Forced task end	`EXEC CICS ABEND`.

Chapter 3. The map DSL

Bricks ships its own line-oriented, BMS-flavoured map DSL (parsed by `mapdsl/`). Maps live as `*.map` files in the directory configured by `maps_dir` (default `runtime/map/`). Each file contains one or more `MAP` … `ENDMAP` blocks. Comments start with `*`.

Format

```
MAP <name> SIZE rowsxcols
  [FIELD AT r,c LEN n <attrs> "literal"]
  [INPUT <fieldname> AT r,c LEN n <attrs> [DEFAULT "value"]]
  [STOP AT r,c]
  [CURSOR AT {fieldname | r,c}]
ENDMAP
```

Statements

`MAP <name> SIZE rowsxcols` The map header. Names must be unique across the directory (case-insensitive); typical sizes are `24x80` (mod 2) and `43x80` (mod 4).

`FIELD AT r,c LEN n <attrs> "literal"` A display-only field that paints the literal at row `r`, column `c`, length `n`. Useful for labels, headings, and panel chrome.

`INPUT <fieldname> AT r,c LEN n <attrs> [DEFAULT "value"]` A named input field. The name is how `SEND MAP FROM(STEM.)` and `RECEIVE MAP INTO(STEM.)` route data to / from this position (`STEM.<fieldname>`). Use `PROT` to make the field write-protected but still named — useful for output values painted by `SEND MAP FROM(STEM.)`.

`STOP AT r,c` An autoskip stop attribute that ends the preceding input field.

`CURSOR AT <fieldname>` *or* **`CURSOR AT r,c`** The home position for the cursor when the map is sent. The named form resolves to `(field.Row, field.Col + 1)` — i.e. one byte to the right of the leading 3270 attribute byte, which is the writable cell. Forward references are allowed; names are resolved after the whole map is parsed. When omitted, the renderer falls back to the first input field.

`ENDMAP` Terminates the map.

Attributes

`PROT`, `UNPROT`, `BRIGHT`, `DIM`, `UNDERSCORE`, `HIDDEN`, `NUMERIC`, `MDT`, `BLINK`, `REVERSE`, `COLOR=BLUE|RED|PINK|GREEN|TURQUOISE|YELLOW|WHITE`.

Catalogue lifecycle

`mapdsl.NewCatalog(dir)` parses every `*.map` file at startup and self-refreshes on subsequent edits. Each `Lookup(name)` stats the directory plus the source file backing the requested name; on `mtime` change the directory is reparsed and swapped atomically. A failed re-parse keeps the prior catalogue in place — the operator can find the broken file with `CEMT PERFORM RESCAN MAP` (see the README).

Example

```
* CICS sign-on screen
MAP CSSN SIZE 24x80
  FIELD AT 1,28 LEN 24 PROT BRIGHT COLOR=TURQUOISE  "BRICKS SIGN-ON"
  FIELD AT 6,15 LEN 9  PROT                         "Userid:"
  INPUT USERID   AT 6,25 LEN 8 UNDERSCORE COLOR=GREEN
  STOP           AT 6,34
  FIELD AT 8,15 LEN 9  PROT                         "Password:"
  INPUT PASSWORD AT 8,25 LEN 8 HIDDEN UNDERSCORE COLOR=RED
  STOP           AT 8,34
  CURSOR         AT USERID
ENDMAP
```

Part 2. EXEC CICS command reference

This part documents every `EXEC CICS` command bricks implements. Each command page follows the same layout: **Format**, **Description**, **Options**, **Conditions**, **Example**.

Commands are grouped by function:

- Chapter 4. Terminal I/O
- Chapter 5. Program control
- Chapter 6. System services
- Chapter 7. KSDS files
- Chapter 8. KSDS browse
- Chapter 9. Temporary storage

Cross-cutting reference:

- Chapter 10. Recovery and condition handling
- Chapter 11. The EIB
- Chapter 12. Response codes
- Chapter 13. Commands not implemented

Chapter 4. Terminal I/O commands

SEND MAP

Send a BMS-style map to the 3270 terminal and wait for the operator to press an AID key.

Format

```
EXEC CICS SEND MAP(name)
              [FROM(stem.)]
              [ERASE]
              [CURSOR(position)]
END-EXEC
```

Description Loads the named map from the catalogue, populates each named field with the value of `<stem>.<fieldname>` (when `FROM` is supplied), paints the screen, and waits for the operator to press an AID key (ENTER, PFn, PAn, CLEAR). On return, the captured response is stored on the TCB so a subsequent `RECEIVE MAP` can pull modified field values back into the program. `EIBAID` and `EIBCPOSN` are updated with the AID character and 1-based cursor position.

If `FROM` is omitted, the map renders using its `INPUT DEFAULT` values. If `FROM` is a literal string, every named field on the map is filled with that literal (rare; supported for parity with BMS).

`ERASE` clears the screen first; without `ERASE` existing fields stay behind underneath the new map (3270 `NoClear`).

Options **MAP(name)** — *required* Name of a map in the catalogue (case-insensitive). MAPFAIL if absent.

FROM(stem.) Stem variable whose tails name the map's input fields. The stem's trailing dot is optional: `FROM(SCR)` and `FROM(SCR.)` are equivalent.

ERASE Clears the screen before painting.

CURSOR(position) 1-based cursor position. Overrides the map's `CURSOR AT` clause.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Map sent, AID received.
MAPFAIL	36	Named map not found in the catalogue.
IOERR	17	The terminal write failed.

Example

```
SCR.GREETING = 'Hello, ' || USR
EXEC CICS SEND MAP('HELO1') FROM(SCR.) ERASE END-EXEC
```

RECEIVE MAP

Retrieve the operator's input from the most recent `SEND MAP`.

Format

```
EXEC CICS RECEIVE MAP(name)
                  [INTO(stem.)]
END-EXEC
```

Description Pulls the response stored by the most recent `SEND MAP` into a stem, one tail per named field on the map (`<stem>.<fieldname>`). When `INTO` is omitted, the default stem is `MAP.`.

A `RECEIVE MAP` with no matching prior `SEND MAP` returns `MAPFAIL`.

Options **MAP(name)** — *required* The same map name passed to the prior `SEND MAP`.

INTO(stem.) Destination stem. Default `MAP.`.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Fields retrieved.
MAPFAIL	36	No prior matching `SEND MAP`, or named map not found.

Example

```
EXEC CICS RECEIVE MAP('CUST1') INTO(IN.) END-EXEC
ACTION = IN.ACTION
CKEY   = IN.CUSTNO
```

CONVERSE

A single verb that fuses `SEND MAP` and `RECEIVE MAP` against the same map. Semantically identical to issuing the two halves back-to-back; provided because real-world CICS programs written between the 1970s and the 1990s used `CONVERSE` heavily, and porting them shouldn't require a SEND/RECEIVE rewrite.

Format

```
EXEC CICS CONVERSE MAP(name) [MAPSET(set)]
                   [FROM(area) | FROMMAP(area)]
                   [INTO(area)]
                   [ERASE] [CURSOR(pos)]
END-EXEC
```

- `FROM(area)` and `FROMMAP(area)` are accepted spellings for the outbound stem; either works, both feed the SEND half.
- `INTO(area)` is the inbound stem; it feeds the RECEIVE half. May be omitted only if the program follows up with a separate `RECEIVE MAP` (rare; using CONVERSE without INTO is technically legal but defeats the point).
- `MAPSET`, `ERASE`, `CURSOR` pass through to both halves.

Behaviour CONVERSE issues `SEND MAP` first, then `RECEIVE MAP` against the same `MAP(name)`. If the SEND half returns anything other than `RESP-NORMAL`, the RECEIVE half is **skipped** and the SEND's response code is returned — matches real CICS semantics (no screen ever made it to the terminal, so there's no operator response to wait for).

Example

```
EXEC CICS CONVERSE MAP('TIM1') FROM(SCR) INTO(SCR)
                   ERASE
END-EXEC.
IF EIBAID = PF03 THEN
    EXEC CICS RETURN END-EXEC
END-IF.
```

This replaces the longer pair:

```
EXEC CICS SEND MAP('TIM1') FROM(SCR) ERASE END-EXEC.
EXEC CICS RECEIVE MAP('TIM1') INTO(SCR)     END-EXEC.
```

Live example: `runtime/cobol/timc.cob` uses CONVERSE for both the input screen and the reminder screen.

SEND TEXT

Free-form text output without a BMS map.

Format

```
EXEC CICS SEND TEXT FROM(area)
                    [LENGTH(n)]
                    [ERASE]
END-EXEC
```

Description Real-CICS `SEND TEXT`. The body is treated as a flat row-major buffer: every `cols` bytes (default 80) lands on the next row, starting at row 0. **3270 has no LF/CR**, so programs must pad each logical line to the column width and concatenate (REXX `LEFT(s,80)`, COBOL `PIC X(80)` group children).

`LENGTH(n)` truncates the body to `n` bytes; bytes past `rows*cols` are dropped. `ERASE` clears the screen before painting; without it, existing fields stay behind. Like `SEND MAP`, the call paints synchronously and waits for an AID, so the operator has time to read the screen before `RETURN`.

Works identically from REXX and COBOL.

Options **FROM(area)** — *required* The buffer to paint. Can be a REXX variable or a COBOL group item.

LENGTH(n) Truncate the body to this length.

ERASE Clear the screen first.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Body painted, AID received.
INVREQ	16	`FROM` missing.
IOERR	17	The terminal write failed.

Example See Chapter 29. Worked examples, the `GETC` program, for the canonical multi-row `SEND TEXT` pattern.

RECEIVE

Retrieve the unedited terminal line typed by the operator at the blank prompt — typically used to pick up command-line arguments.

Format

```
EXEC CICS RECEIVE INTO(buffer)
                  [LENGTH(len)]
END-EXEC
```

Description Returns the unedited terminal line the operator typed at the blank prompt (TRANSID prefix included), e.g. `EXAM 1 2 3`. Single-shot per task: a second `RECEIVE` in the same task, or any `RECEIVE` in a chained `RETURN TRANSID` task, returns `EOC` (RESP=6).

When `LENGTH(var)` is a bare variable, the actual byte count of the line is written back. The receiving COBOL field's `PIC X(n)` width handles padding/truncation via standard `MOVE` semantics; programs detect truncation by comparing `len` against n.

Options **INTO(buffer)** — *required* Destination variable for the line.

LENGTH(len) When a bare variable, receives the actual length on return.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Line returned.
EOC	6	No input available (already consumed in this task chain).

Example

```
EXEC CICS RECEIVE INTO(BUF) LENGTH(LEN) END-EXEC   /* "GETC 100" */
PARSE VAR BUF TID CKEY .
```

```
EXEC CICS RECEIVE INTO(WS-INPUT) LENGTH(WS-LEN) END-EXEC
UNSTRING WS-INPUT DELIMITED BY ' '
   INTO WS-TID WS-A WS-B WS-C
END-UNSTRING.
```

Chapter 5. Program control commands

RETURN

End the current task; optionally chain to another TRANSID with a COMMAREA.

Format

```
EXEC CICS RETURN [TRANSID(id)]
                 [COMMAREA(data)]
END-EXEC
```

Description Sets `tcb.NextTransid` and `tcb.Commarea` and ends the task. With no `TRANSID`, control falls back to the blank prompt. With a `TRANSID`, the dispatcher invokes that TRANSID next, with the supplied `COMMAREA` available to the new task as `DFHCOMMAREA` and its length as `EIBCALEN`.

Options **TRANSID(id)** The next TRANSID to dispatch.

COMMAREA(data) Bytes to flow into the next task.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Task ended.

Example

```
EXEC CICS RETURN TRANSID('CUST') COMMAREA(SAVEAREA) END-EXEC
```

XCTL

Transfer control to another program. The current program does not resume.

Format

```
EXEC CICS XCTL PROGRAM(name)
               [COMMAREA(data)]
END-EXEC
```

Description Transfer of control. The named program runs in place of the current one; the COMMAREA, if supplied, becomes its `DFHCOMMAREA`. The current task's chain continues with the new program; `RETURN TRANSID` semantics carry through unchanged.

Options **PROGRAM(name)** — *required* The target program. Resolved through `runtime/transactions.conf`.

COMMAREA(data) Bytes to flow to the target.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Control transferred.
PGMIDERR	27	Target program not in `transactions.conf`.
INVREQ	16	`PROGRAM` missing.

Example

```
EXEC CICS XCTL PROGRAM('VALIDATE') COMMAREA(SCR) END-EXEC
```

LINK

Synchronously call a sub-program with bidirectional COMMAREA.

Format

```
EXEC CICS LINK PROGRAM(name)
              [COMMAREA(var | 'literal')]
END-EXEC
```

Description The target program runs in a fresh frame with its `DFHCOMMAREA` preloaded from the caller's `COMMAREA(var)` (or literal). When the sub-program exits, its final `DFHCOMMAREA` is written back to the caller's variable.

Caller state — `NextTransid`, `Commarea`, `LastResponse`, `LastMapName` — is saved before the LINK and restored on return, so the LINK is transparent to the caller's pseudo-conversational context.

The per-transaction ACL is rechecked on the target so a low-privilege caller cannot escalate by linking into an admin program.

REXX and COBOL programs can LINK to each other freely; `DFHCOMMAREA` is marshalled as opaque bytes.

Options **PROGRAM(name)** — *required* The sub-program to call. Resolved through `runtime/transactions.conf`.

COMMAREA(var | 'literal') Bidirectional data area. A bare variable is read on entry and written on exit; a literal is read-only.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Sub-program returned cleanly.
PGMIDERR	27	Target not in `transactions.conf`, sub-program errored, or the caller is not authorised for the target.

Example

```
SAV = ''                                /* working area */
EXEC CICS LINK PROGRAM('CUSV') COMMAREA(SAV) END-EXEC
IF EIBRESP <> 0 THEN SIGNAL ERR
PARSE VAR SAV STATUS '|' MSG
```

ABEND

Abnormally terminate the current task.

Format

```
EXEC CICS ABEND [ABCODE(code)]
                [NODUMP]
END-EXEC
```

Description Ends the task with the supplied 4-character abend code (default AAAA). Bricks logs the abend on the operator console and reflects it in the TxCB's status; pseudo-conversational chains are broken (no `NextTransid` is honoured after an ABEND).

Options **ABCODE(code)** 4-character code to record. Defaults to AAAA.

NODUMP Accepted for parity with IBM CICS; no dumps are produced anyway.

Conditions ABEND does not return; nothing follows it in the program flow.

Example

```
IF EIBRESP <> 0 THEN
   EXEC CICS ABEND ABCODE('FIO1') END-EXEC
```

START

Schedule a transaction to fire on this same terminal after a delay (or at a wall-clock time), optionally passing a byte payload the new task picks up via RETRIEVE. This is bricks's implementation of CICS's classic deferred-work pattern: a task queues "do X in 60 seconds" and ends; the terminal returns to the blank prompt; when the timer elapses, bricks dispatches X automatically.

Format

```
EXEC CICS START TRANSID(name)
                [INTERVAL(hhmmss) | TIME(hhmmss)]
                [FROM(area) [LENGTH(n)]]
                [TERMID(tttt)]
END-EXEC
```

Options **TRANSID(name)** *(required)* 4-character transaction id, must exist in `runtime/transactions.conf`. The dispatcher applies the normal per-transaction ACL at fire time, so the operator must still be authorised to invoke `name`.

INTERVAL(hhmmss) Delay before firing, expressed as up to 6 digits in `HHMMSS` form (left-padded with zeroes). `INTERVAL(30)` = 30 s; `INTERVAL(000130)` = 1 min 30 s; `INTERVAL(010000)` = 1 h. Omit (or pair with `INTERVAL(0)`) to fire on the next prompt-loop iteration.

TIME(hhmmss) Absolute wall-clock target, also `HHMMSS`. Respects the operator's `time_zone=` setting. If the target is earlier than now, bricks rolls forward to the same time tomorrow (real-CICS semantics). Mutually exclusive with `INTERVAL`.

FROM(area) [LENGTH(n)] Payload bytes the new task will pull back via `RETRIEVE`. `LENGTH` truncates `area` to `n` bytes; if omitted, the entire string value of `area` is queued. The bytes are copied at `START` time, so the issuing task may mutate `area` afterwards without disturbing the queued payload.

TERMID(tttt) Must equal the issuing terminal's `EIBTRMID` in this version. Cross-terminal STARTs return `RESP-INVREQ` with "only same-terminal STARTs are supported."

Scope and limitations (v1)

- **Same-terminal only.** A START always fires on the issuing TCB. No-`TERMID` background tasks and `TERMID(other)` are rejected; both require infrastructure (synthetic TCBs, cross-session authorisation) that's deferred.
- **In-memory queue.** Pending STARTs do **not** survive a bricks restart. If you need recovery-protected scheduling you'll have to wait for `START PROTECT` (out of scope).
- **No `REQID`, `SYSID`, `QUEUE`, `PROTECT`, `HOURS`, `MINUTES`, `SECONDS` options.** Each returns `RESP-INVREQ` with the option name in the error string so a port can be triaged.
- **One pending payload per task.** When the START fires, the FROM bytes go into `tcb.RetrieveBuf`. The new task's first `RETRIEVE` drains it; a second `RETRIEVE` returns `RESP-ENDDATA` (29).

Conditions

- `RESP-NORMAL` on successful enqueue.
- `RESP-INVREQ` for missing TRANSID, cross-terminal TERMID, unsupported option, or malformed INTERVAL/TIME.

Example

```
EXEC CICS START TRANSID('TIMC') INTERVAL('000030')
                FROM(MSG)
END-EXEC.
```

Schedules `TIMC` to fire on this terminal in 30 seconds with the contents of `MSG` queued as the payload. See `runtime/cobol/timc.cob` and `runtime/rexx/timr.rexx` for the worked examples.

RETRIEVE

Pull back the `FROM(...)` payload of the `START` that scheduled the *current* task. RETRIEVE is the receiving half of the START / RETRIEVE pair; the first thing a transaction fired by `START` typically does.

Format

```
EXEC CICS RETRIEVE INTO(var) [LENGTH(lenvar)] END-EXEC
```

Options **INTO(var)** *(required)* Variable / DATA area to receive the payload bytes. Must be a name reference, not a string literal.

LENGTH(lenvar) Optional. The variable named here is written with the actual byte length of the payload after a successful read, so the caller can size the consumer code correctly.

Behaviour

- Cold dispatch (operator typed the TRANSID at the prompt) □ `RESP-ENDDATA` (29), nothing written to `INTO`.
- Scheduled re-entry □ `RESP-NORMAL`, payload bytes copied into `INTO`, `tcb.RetrieveBuf` cleared.
- Second RETRIEVE in the same task □ `RESP-ENDDATA` (the buffer is cleared on first read; v1 supports one pending payload per task).

Idiom The canonical START / RETRIEVE program tests `EIBRESP` first to discover *which dispatch path* it's on, then branches:

```
MOVE SPACES TO BUF.
EXEC CICS RETRIEVE INTO(BUF) END-EXEC.

IF EIBRESP = RESP-NORMAL THEN
    PERFORM SHOW-REMINDER      *> scheduled fire
    EXEC CICS RETURN END-EXEC
END-IF.

*> Cold path: prompt the operator, schedule next fire.
PERFORM PROMPT-AND-SCHEDULE.
```

REXX equivalent in `runtime/rexx/timr.rexx`; COBOL in `runtime/cobol/timc.cob`.

Chapter 6. System services

ASSIGN

Read EIB, session, and environment fields into the program.

Format

```
EXEC CICS ASSIGN <FIELD>(target) [<FIELD>(target) ...]
END-EXEC
```

Description Reads one or more system fields into program variables. Multiple options can be combined in a single `ASSIGN` call. Each `<FIELD>` is one of the keywords below; `target` is the variable that receives the value.

Options

Option	Returns
`USERID(t)`	Signed-on userid (empty until CSSN succeeds).
`TERMID(t)` / `EIBTRMID(t)`	Unique 4-digit terminal id (`T0001` …).
`EIBAID(t)`	Single-byte AID character of the most recent `SEND MAP` / `RECEIVE MAP`. Compare with `C2X(EIBAID) = 'F3'` (REXX) or `EIBAID = X'F3'` (COBOL) to detect PF3, etc.
`EIBCPOSN(t)`	1-based cursor position from the most recent map response.
`EIBCALEN(t)`	Length of `DFHCOMMAREA` flowed in from the caller.
`TWALENG(t)` / `TCTUALENG(t)`	Always `0` (bricks does not allocate a TWA / TCTUA).
`SCREENHT(t)` / `SCREENWD(t)`	Negotiated terminal rows / columns.
`ALTSCRNHT(t)` / `ALTSCRNWD(t)`	Same values; bricks does not distinguish primary and alternate sizes.
`CONNECTED(t)` / `CONNTIME(t)`	Wall-clock connect timestamp `YYYY-MM-DD HH:MM:SS`.
`TLS(t)`	`yes` when the session is on the TLS listener, `no` otherwise.
`DATE(t)`	Today as `YYYYMMDD`. *(bricks-specific)*
`TIME(t)`	Now as `HHMMSS`. *(bricks-specific)*
`TODAYYR(t)` / `TODAYMO(t)` / `TODAYDY(t)`	Today's year / month / day individually. *(bricks-specific)*
`DAYCOUNT(t)`	Days since 1970-01-01. Subtract two values for an exact day delta. *(bricks-specific)*

The bricks-specific options exist primarily so the COBOL subset can do date math without REXX-style intrinsic functions; see `runtime/cobol/qagc.cob` for an example that computes age in years from a birthdate.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	All requested fields returned.
INVREQ	16	A field name is unknown, or its target is a literal rather than a variable.

Example

```
EXEC CICS ASSIGN USERID(USR)
                 TERMID(TRM)
                 SCREENHT(SCRH)
                 SCREENWD(SCRW)
END-EXEC
```

ASKTIME

Refresh the EIB date/time fields and, optionally, return the current time as a 15-digit absolute timestamp (`ABSTIME`).

Format

```
EXEC CICS ASKTIME [ABSTIME(target)]
END-EXEC
```

Description `ASKTIME` re-reads the system clock and writes the current date and time into `EIBDATE` and `EIBTIME`. Both fields follow the IBM CICS formats:

- `EIBDATE` — packed-style decimal `0CYYDDD`, where `C` is `(century - 19)` and `YYDDD` is the two-digit year and Julian day-of-year. For 2026-05-12 (Julian day 132), `EIBDATE = 1026132`.
- `EIBTIME` — six-digit `HHMMSS`.

If `ABSTIME(target)` is given, `target` receives a 15-digit decimal string representing milliseconds since `1900-01-01 00:00:00.000`. Pass that value to `FORMATTIME` to break it back into formatted date / time components.

`ASKTIME` is the only way to get a fresh `ABSTIME`; the bricks-specific `ASSIGN DATE` / `TIME` shortcuts return formatted strings only.

Options **ABSTIME(target)** Variable that receives the 15-character absolute time. Optional; when omitted, only `EIBDATE` and `EIBTIME` are refreshed.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Always (clock reads cannot fail).

Example

```
EXEC CICS ASKTIME ABSTIME(NOW) END-EXEC
EXEC CICS FORMATTIME ABSTIME(NOW)
                     YYYYMMDD(TODAY)
                     TIME(CLOCK) TIMESEP(':')
END-EXEC
SAY 'Stamped at' TODAY CLOCK
```

FORMATTIME

Decode an `ABSTIME` value into formatted date and time fields.

Format

```
EXEC CICS FORMATTIME ABSTIME(source)
                     [DATE(t)] [DATEFORM(fmt)] [DATESEP(c)]
                     [YYYYMMDD(t)] [MMDDYYYY(t)] [DDMMYYYY(t)]
                     [MMDDYY(t)]   [DDMMYY(t)]
                     [YYYYDDD(t)]  [YYDDD(t)]
                     [TIME(t)] [TIMESEP(c)]
                     [YEAR(t)] [MONTHOFYEAR(t)] [DAYOFMONTH(t)]
                     [DAYOFWEEK(t)] [DAYCOUNT(t)]
END-EXEC
```

Description `FORMATTIME` is the IBM-standard companion to `ASKTIME`. Given a 15-digit `ABSTIME` value (typically returned by `ASKTIME ABSTIME(...)`, but any 15-digit decimal millisecond stamp is accepted), it writes the requested representations into the named target variables.

Any combination of output options may be requested in a single call; options that are omitted cost nothing.

Options **ABSTIME(source)** *(required)* The 15-digit decimal absolute time to decode.

DATE(t) with optional **DATEFORM(fmt)** and **DATESEP(c)** Generic date target. `DATEFORM` is one of `YYYYMMDD` (default), `MMDDYYYY`, `DDMMYYYY`, `MMDDYY`, `DDMMYY`. `DATESEP(c)` inserts a one-character separator between components (e.g. `'-' □ 2026-05-12`); omit `DATESEP` for a packed string with no separators.

YYYYMMDD / MMDDYYYY / DDMMYYYY / MMDDYY / DDMMYY Direct date format targets; each respects `DATESEP(c)` if given.

YYYYDDD(t) / YYDDD(t) Julian-style date with three-digit day-of-year. Honour `DATESEP`.

TIME(t) with optional **TIMESEP(c)** Six-digit `HHMMSS`, or with `TIMESEP(':')` formatted as `HH:MM:SS`.

YEAR(t) — four-digit year (`2026`). **MONTHOFYEAR(t)** — 1..12. **DAYOFMONTH(t)** — 1..31. **DAYOFWEEK(t)** — `0` (Sunday) .. 6 (Saturday), per IBM CICS. **DAYCOUNT(t)** — days since `1900-01-01` (signed, integer).

Conditions

Condition	EIBRESP	Cause
NORMAL	0	All requested fields written.
INVREQ	16	`ABSTIME` is missing, not 15 decimal digits, or out of range.

Example

```
EXEC CICS ASKTIME ABSTIME(WS-NOW) END-EXEC
EXEC CICS FORMATTIME ABSTIME(WS-NOW)
                     DATE(WS-DATE)  DATEFORM('DDMMYYYY')  DATESEP('/')
                     TIME(WS-TIME)  TIMESEP(':')
                     DAYOFWEEK(WS-DOW)
END-EXEC.
*  WS-DATE → "12/05/2026"     WS-TIME → "10:30:45"     WS-DOW → "2"
```

Chapter 7. KSDS file commands

These commands operate on a **single record**, identified by a key, in a CICS FILE. Each FILE is a bbolt bucket inside `data/files.boltdb`; record bodies are opaque bytes (the application chooses the layout). See *How file storage works* in the README for the on-disk model.

READ

Read a record from a KSDS by key.

Format

```
EXEC CICS READ FILE(name)
              {INTO(target) | SET(target)}
              RIDFLD(key)
              [UPDATE]
              [LENGTH(len)]
END-EXEC
```

Description B+tree exact-key lookup, O(log n). The record bytes (whatever the application stored) come back into the target. `UPDATE` records a per-session lock on the key, gating a subsequent `REWRITE` on the same FILE.

When `LENGTH(var)` is a bare variable, the actual record length is written back.

Options **FILE(name)** — *required* The CICS FILE name.

INTO(target) / **SET(target)** — *one required* Destination for the record body. `INTO` and `SET` are accepted interchangeably here.

RIDFLD(key) — *required* The key to look up.

UPDATE Record a per-session lock so a subsequent `REWRITE` can update this record.

LENGTH(len) Receives the record's actual length when the option is a bare variable.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Record returned.
NOTFND	13	Key not present.
INVREQ	16	Missing required option, or invalid file name.
IOERR	17	Underlying store error.

Example

```
EXEC CICS READ FILE('CUSTOMERS')
              INTO(REC) RIDFLD(CKEY)
END-EXEC
IF EIBRESP = 13 THEN MSG = 'Customer not found'
```

WRITE

Insert a new record into a KSDS.

Format

```
EXEC CICS WRITE FILE(name)
               FROM(data)
               RIDFLD(key)
END-EXEC
```

Description B+tree insert. The bucket for the FILE is created implicitly on first WRITE — there is no `EXEC CICS DEFINE FILE` step. `DUPREC` if a record with the same key already exists.

The write commits in a single bbolt `Update` transaction with `fsync` on success.

Options **FILE(name)** — *required* Target FILE. Created if absent.

FROM(data) — *required* The record bytes to store.

RIDFLD(key) — *required* The key under which to store the record.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Record written.
DUPREC	14	A record with this key already exists.
INVREQ	16	Missing required option, or invalid name.
IOERR	17	Underlying store error.

Example

```
REC = NM || '|' || AD || '|' || CY || '|' || PH
EXEC CICS WRITE FILE('CUSTOMERS') FROM(REC) RIDFLD(CKEY) END-EXEC
```

REWRITE

Replace the record locked by the most recent `READ … UPDATE`.

Format

```
EXEC CICS REWRITE FILE(name)
                  FROM(data)
END-EXEC
```

Description Overwrites the value at the key locked by the most recent `READ FILE … UPDATE` on the same FILE. Releases the per-FCB update lock at end of transaction.

Options **FILE(name)** — *required* **FROM(data)** — *required*

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Record updated.
INVREQ	16	No prior `READ … UPDATE`, or the lock has been released.
IOERR	17	Underlying store error.

Example

```
EXEC CICS READ FILE('CUSTOMERS') INTO(REC) RIDFLD(CKEY) UPDATE END-EXEC
PARSE VAR REC NM '|' AD '|' CY '|' PH
PH = NEWPHONE                                       /* mutate */
REC = NM || '|' || AD || '|' || CY || '|' || PH
EXEC CICS REWRITE FILE('CUSTOMERS') FROM(REC) END-EXEC
```

DELETE

Remove a record from a KSDS.

Format

```
EXEC CICS DELETE FILE(name)
                [RIDFLD(key)]
END-EXEC
```

Description If `RIDFLD` is supplied, deletes that key. Otherwise deletes the key locked by the most recent `READ … UPDATE` on the same FILE.

Options **FILE(name)** — *required*

RIDFLD(key) Optional. When omitted, the most recent READ-UPDATE key is used.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Record deleted.
NOTFND	13	Key not present.
INVREQ	16	No `RIDFLD` and no prior READ-UPDATE; or invalid name.
IOERR	17	Underlying store error.

Example

```
EXEC CICS DELETE FILE('CUSTOMERS') RIDFLD(CKEY) END-EXEC
```

Chapter 8. KSDS browse commands

A **browse** walks a CICS FILE in B+tree key order. The browse runs inside a bbolt MVCC read transaction, so the cursor sees a stable point-in-time snapshot — concurrent `WRITE` / `REWRITE` / `DELETE` on the same FILE do not disturb an in-progress browse.

The browse is **per-task** and tied to one FILE. A program may have multiple browses open on different FILEs at once; a second `STARTBR` on the same FILE replaces the first (`STARTBR` is implicitly idempotent in CICS).

The dispatcher releases any cursor the program forgot to `ENDBR` via a `defer handler.CloseBrowses()` at task end.

STARTBR

Open a browse cursor on a KSDS file.

Format

```
EXEC CICS STARTBR FILE(name)
                  [RIDFLD(start)]
                  [GTEQ | EQUAL]
                  [GENERIC]
                  [KEYLENGTH(n)]
END-EXEC
```

Description Opens a B+tree browse cursor on the file. With no `RIDFLD`, positions on the first key. With `RIDFLD` and `GTEQ` (the IBM default and bricks default), positions on the first key ≥ start. With `EQUAL`, requires an exact match (`NOTFND` if absent). With `GENERIC` and `KEYLENGTH(n)`, positions on (and walks only through) keys whose first n bytes match the first n bytes of `RIDFLD`.

Options **FILE(name)** — *required*

RIDFLD(start) Starting key. Default: first key in the file.

GTEQ | **EQUAL** Comparison rule. Default `GTEQ`.

GENERIC Restrict the walk to keys whose prefix matches.

KEYLENGTH(n) With `GENERIC`, the prefix length to match.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Cursor opened.
NOTFND	13	`EQUAL` requested but no such key.
INVREQ	16	Missing required option, invalid name, etc.
IOERR	17	Underlying store error.

Example

```
EXEC CICS STARTBR FILE('CUSTOMERS')
                  RIDFLD('NY-')
                  GENERIC KEYLENGTH(3)
END-EXEC
```

READNEXT

Step the open browse cursor forward by one record.

Format

```
EXEC CICS READNEXT FILE(name)
                   {INTO(target) | SET(target)}
                   [RIDFLD(keyvar)]
                   [LENGTH(len)]
END-EXEC
```

Description Advances the cursor and returns the next record (or the first record if this is the first READNEXT after STARTBR). When RIDFLD is a bare variable, the matching key is written back; when LENGTH is a bare variable, the actual record length is written back.

Returns ENDFILE past the last key (or past the end of the GENERIC prefix). Records that were deleted by a concurrent transaction between STARTBR and the read are skipped automatically (bounded forward loop, no goroutine-stack risk).

Options **FILE(name)** — *required*

INTO(target) / **SET(target)** — *one required*

RIDFLD(keyvar) Receives the key of the record returned.

LENGTH(len) Receives the actual record length.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Record returned.
ENDFILE	20	Past the last key (or out of GENERIC prefix).
INVREQ	16	No STARTBR is open on this FILE.
IOERR	17	Underlying store error.

Example See STARTBR example, then:

```
DO FOREVER
  EXEC CICS READNEXT FILE('CUSTOMERS') INTO(REC) RIDFLD(K) END-EXEC
  IF EIBRESP = 20 THEN LEAVE
  SAY K ':' REC
END
```

READPREV

Step the open browse cursor backward by one record.

Format

```
EXEC CICS READPREV FILE(name)
                   {INTO(target) | SET(target)}
                   [RIDFLD(keyvar)]
                   [LENGTH(len)]
END-EXEC
```

Description Same writeback rules as READNEXT. Returns ENDFILE before the first key (or before the start of the GENERIC prefix). Useful for paginating backward through a key range.

Options As for READNEXT.

Conditions As for READNEXT, with ENDFILE meaning *before the first key*.

Example

```
EXEC CICS READPREV FILE('CUSTOMERS') INTO(REC) RIDFLD(K) END-EXEC
```

RESETBR

Reposition the cursor of an open browse without closing the underlying read transaction.

Format

```
EXEC CICS RESETBR FILE(name)
                  RIDFLD(start)
                  [GTEQ | EQUAL]
                  [GENERIC]
                  [KEYLENGTH(n)]
END-EXEC
```

Description Cheaper than ENDBR + STARTBR when the program wants to jump within the same browse session — the read snapshot is preserved.

Options As for STARTBR. RIDFLD is required.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Cursor repositioned.
INVREQ	16	No STARTBR is open on this FILE.
NOTFND	13	EQUAL requested but no such key.

Example

```
EXEC CICS RESETBR FILE('CUSTOMERS') RIDFLD('00500') END-EXEC
```

ENDBR

Close an open browse cursor.

Format

```
EXEC CICS ENDBR FILE(name)
END-EXEC
```

Description Releases the cursor and the underlying bbolt read transaction. The dispatcher releases any cursor the program forgot to ENDBR at task end, but explicit ENDBR is recommended.

Options **FILE(name)** — *required*

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Cursor closed.
INVREQ	16	No STARTBR is open on this FILE.

Example

```
EXEC CICS ENDBR FILE('CUSTOMERS') END-EXEC
```

Chapter 9. Temporary storage and transient data commands

Bricks supports two related queue families:

- **Temporary Storage (TS)** — ordered, persistent sequences of opaque byte items inside the bbolt database. Each queue is a sub-bucket whose keys are 8-byte big-endian item numbers (1, 2, 3 …) and whose values are the item payloads. Item-addressed; reads and writes are O(log N). Use for ephemeral state, scratch storage, producer/consumer chaining inside the database.

- **Transient Data extra-partition (TD)** — sequential text files in a sandboxed on-disk directory (`tmp_dir`). Line-oriented; read-once-and-advance / append-only semantics. Use for staging imports from text files into VSAM and exports out of it. The REXX `LINEIN` / `LINEOUT` / `STREAM` family hits the same backend, so a file written by COBOL is readable by REXX and vice-versa.

The encoding contract for TD files is strict: ASCII bytes only (`0x09` TAB, `0x20`–`0x7E` printable), no EBCDIC, no UTF; lines are terminated by a single LF (`0x0A`); CR (`0x0D`) is rejected on write with `INVREQ`. The sandbox enforces a flat namespace under `tmp_dir` — no sub-directories, no leading dot, no `..`, no slashes in the queue name. See The `tmp_dir` sandbox below for the full rules.

READQ TS

Read an item from a temporary storage queue.

Format

```
EXEC CICS READQ TS QUEUE(name)
                   INTO(target)
                   [ITEM(n) | NEXT]
                   [LENGTH(len)]
                   [NUMITEMS(num)]
END-EXEC
```

`QNAME` is accepted as a synonym for `QUEUE`.

Description Reads one item from the queue. With `ITEM(n)`, returns item n. Without `ITEM`, or with `NEXT`, advances the **per-task implicit cursor**: the first cursor-less READQ on a queue returns item 1, the second returns item 2, and so on. The cursor is keyed on the running TxCB and released when the task ends — a fresh invocation of the same TRANSID starts at item 1 again.

Options **QUEUE(name) / QNAME(name)** — *required*

INTO(target) — *required*

ITEM(n) | **NEXT** Item to read. Default = next per the implicit cursor.

LENGTH(len) Receives the actual item length.

NUMITEMS(num) Receives the queue's current high-water item count.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Item returned.
ITEMERR	26	`ITEM(n)` out of range, or implicit cursor past the last item.
QIDERR	44	Queue does not exist.

Condition	EIBRESP	Cause
INVREQ	16	Missing required option, invalid name.

Example

```
DO FOREVER
  EXEC CICS READQ TS QUEUE(QNM) INTO(REC) END-EXEC
  IF EIBRESP = 26 THEN LEAVE                /* end of queue */
  SAY REC
END
```

WRITEQ TS

Append a new item to a queue, or rewrite an existing item.

Format

```
EXEC CICS WRITEQ TS QUEUE(name)
                   FROM(data)
                   [ITEM(n) REWRITE]
END-EXEC
```

`QNAME` is accepted as a synonym for `QUEUE`.

Description In **append** mode (no `ITEM REWRITE`), the item is added at the next sequence number; an in-memory high-water-mark counter assigns the number without scanning. When `ITEM(var)` is a bare variable, the assigned item number is written back.

In **rewrite** mode (both `ITEM(n)` and `REWRITE` present), item n is replaced.

The write commits in a single bbolt transaction so a crash mid-write leaves either the prior state or the new one — never a partial.

Options **QUEUE(name)** / **QNAME(name)** — *required* **FROM(data)** — *required* **ITEM(n) REWRITE** Both required for rewrite mode; either alone is rejected.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Item written.
ITEMERR	26	`ITEM(n) REWRITE` and item n does not exist.

Condition	EIBRESP	Cause
INVREQ	16	Missing required option, invalid name, or `ITEM`/`REWRITE` not paired.
IOERR	17	Underlying store error.

Example

```
EXEC CICS WRITEQ TS QUEUE('AUDIT') FROM(MSG) ITEM(I) END-EXEC
SAY 'Wrote item' I
```

DELETEQ TS

Delete an entire temporary storage queue.

Format

```
EXEC CICS DELETEQ TS QUEUE(name) END-EXEC
```

Description Drops the queue's sub-bucket and resets the in-memory counters and cursors. Subsequent `WRITEQ` will recreate it starting at item 1.

Options **QUEUE(name)** / **QNAME(name)** — *required*

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Queue deleted.
QIDERR	44	Queue does not exist.

Example

```
EXEC CICS DELETEQ TS QUEUE('AUDIT') END-EXEC
```

READQ TD

Read the next line from a sequential text file in `tmp_dir`.

Format

```
EXEC CICS READQ TD QUEUE(name)
                   INTO(target)
                   [LENGTH(len)]
END-EXEC
```

`QNAME` is accepted as a synonym for `QUEUE`. `name` is a flat filename under `tmp_dir`; sub-directories and traversal are rejected with `INVREQ`.

Description Reads the next LF-terminated line from `tmp_dir/name`, strips the terminating LF, and copies the payload into `target`. The file is auto-opened in read mode on the first call per task and the read cursor advances item-by-item across subsequent calls. A program that WRITEs then READs the same queue causes the handler to close the write handle and reopen for read; the read cursor restarts at line 1. The handle is closed automatically at task end.

Options **QUEUE(name) / QNAME(name)** — *required*

INTO(target) — *required*

LENGTH(len) Receives the line's byte count (after the LF strip).

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Line returned.
QZERO	12	End-of-file. `target` is untouched; bricks closes the handle so a later READQ on the same queue rewinds to line 1.
QIDERR	44	File does not exist in `tmp_dir`.
INVREQ	16	Missing `INTO` / invalid name / sandbox rejection.
IOERR	17	Underlying file-system error.

Example

```
PERFORM IMPORT-ONE UNTIL DONE-FLAG = 'Y'.
...
IMPORT-ONE.
    MOVE SPACES TO REC.
    EXEC CICS READQ TD QUEUE('orders.sample.txt') INTO(REC) END-EXEC.
    IF EIBRESP = 12 THEN
        MOVE 'Y' TO DONE-FLAG
    END-IF.
```

```
    IF EIBRESP = 0 THEN
        PERFORM HANDLE-RECORD
    END-IF.
```

WRITEQ TD

Append a line to a sequential text file in `tmp_dir`.

Format

```
EXEC CICS WRITEQ TD QUEUE(name)
                    FROM(data)
                    [LENGTH(len)]
END-EXEC
```

`QNAME` is accepted as a synonym for `QUEUE`.

Description Appends `data` plus a single LF to `tmp_dir/name`. The file is auto-opened in append mode (creating it if absent) on the first call per task. Trailing ASCII spaces on `data` are right-stripped before write — COBOL `PIC X(n)` values arrive padded to the right and the canonical text-file convention is to strip them. The payload is validated byte-by-byte: anything outside `0x09` (TAB), `0x0A` (LF), or `0x20–0x7E` (printable ASCII) causes the write to fail with `INVREQ` and a byte-offset diagnostic. CR (`0x0D`) is explicitly rejected — bricks emits Unix-style LF-only files.

Options **QUEUE(name)** / **QNAME(name)** — *required*

FROM(data) — *required*

LENGTH(len) Accepted for syntactic compatibility; the actual byte count is `LENGTH(data)` after rstrip.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Line appended.
INVREQ	16	Missing `FROM` / invalid name / sandbox rejection / non-ASCII byte / CR in payload.
IOERR	17	Underlying file-system error.

Example

```
DO I = 1 TO REC.0
   EXEC CICS WRITEQ TD QUEUE('export.txt') FROM(REC.I) END-EXEC
END
```

DELETEQ TD

Delete a sequential text file in `tmp_dir`.

Format

```
EXEC CICS DELETEQ TD QUEUE(name) END-EXEC
```

Description Closes any handle the task has open on `name`, then unlinks the file. Already-gone is treated as success (matches the `DELETE FILE` forgiving semantics in bricks). Use to reset an export staging file before a fresh batch.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	File deleted (or already absent).
INVREQ	16	Invalid name / sandbox rejection.
IOERR	17	Underlying file-system error.

Example

```
EXEC CICS DELETEQ TD QUEUE('export.txt') END-EXEC.
```

The `tmp_dir` sandbox

`tmp_dir` is configured in `bricks.cnf`:

```
tmp_dir = runtime/tmp
```

Defaults to `runtime/tmp` (under `runtime_dir`) when the line is omitted. The directory is created at startup if missing.

Name rules. A queue name must match `[A-Za-z0-9._-]{1,255}` — no leading dot, no `..`, no slash, no backslash, no NUL. Bricks also runs `filepath.Rel` against every resolved path as defense-in-depth, so symlink shenanigans bounce too.

Encoding rules. ASCII only — no EBCDIC, no UTF-8, no UTF-16. The write path validates every byte and rejects with `INVREQ` on the first violation, naming the offending offset and hex value. Lines terminate with LF only; CR is rejected on write. The read path splits on LF and preserves any CR bytes it finds verbatim — bricks does **not** silently strip them. If you need to import a file authored on Windows, pre-strip the CRs:

```
tr -d '\r' < windows.csv > runtime/tmp/clean.csv
```

Cross-language interop. Both COBOL (`READQ TD` / `WRITEQ TD`) and REXX (`LINEIN` / `LINEOUT` / `STREAM`) hit the same `*cics.TmpStore` backend. A file produced by one language is readable by the other; the only constraint is that both sides agree on the column format inside each line (canonical bricks convention: pipe-delimited).

Task-end cleanup. Every handle the program opens is closed automatically when the task ends. A program that forgets to call `STREAM CLOSE` or `DELETEQ TD` does not leak descriptors.

Chapter 10. Recovery and condition handling

This chapter covers two related families:

- **Unit-of-work commands** — `SYNCPOINT` and `SYNCPOINT ROLLBACK` group multiple file / TS mutations into one atomic step that can be committed or undone.
- **Non-local control commands** — `HANDLE CONDITION`, `IGNORE CONDITION`, `HANDLE AID`, and `HANDLE ABEND` arm program labels that the dispatcher branches to on a non-`NORMAL` response, on a particular AID key, or when the task abends. Together with the per-call `EIBRESP` test (Chapter 12) and REXX `SIGNAL ON ERROR` (Chapter 18) they cover the full range of CICS error-handling idioms.

How the unit of work works

Each task owns a **journal** of pending undo entries that lives on its `TxCB`. Every successful `WRITE` / `REWRITE` / `DELETE` and every `WRITEQ TS` / `DELETEQ TS` appends an inverse operation to the journal *immediately after* the underlying bbolt write commits.

- `SYNCPOINT` clears the journal — the work becomes irrevocably committed, and a new unit of work begins.
- `SYNCPOINT ROLLBACK` walks the journal in reverse and applies each inverse op (re-writing pre-images, re-creating deleted records, restoring queue contents, etc.), then clears the journal.
- `RETURN` performs an **implicit `SYNCPOINT`** before the task ends.
- `ABEND` performs an **implicit `SYNCPOINT ROLLBACK`** *unless* a `HANDLE ABEND` exit catches it; the exit may then choose whether to commit or roll back explicitly.

In-flight uncommitted writes are visible to concurrent tasks (this matches IBM CICS under `READ NO UPDATE`); the rollback model exists to recover *the current task* from its own partially-applied work, not to provide multi-task isolation.

How the condition / AID / abend traps work

After every `EXEC CICS` command, the dispatcher writes `EIBRESP` and then consults three optional per-task tables:

Table	Populated by	Consulted	Branches when
Condition map	`HANDLE CONDITION` / `IGNORE CONDITION`	After every command	`EIBRESP ≠ NORMAL` and a label is armed for that condition (or for `ERROR`).
AID map	`HANDLE AID`	After every command that updates `EIBAID` (`SEND MAP`, `RECEIVE MAP`, `RECEIVE`)	The new `EIBAID` matches an armed key (`PF1–PF24`, `PA1–PA3`, `ENTER`, `CLEAR`, or the catch-all `ANYKEY`).
Abend exit	`HANDLE ABEND`	When the task abends	An exit is armed and not cancelled.

When a trap fires, control transfers to the named label as if the program had `GO TO`'d (COBOL) or `SIGNAL`'d (REXX) it directly. The trap *stays armed* across the branch — re-arming after each fire is not required (and the `IGNORE` / bare-name disarm forms exist for when the program wants the trap to stop firing).

SYNCPOINT

Commit the current unit of work and begin a new one.

Format

```
EXEC CICS SYNCPOINT END-EXEC
```

Description Clears the task's undo journal. All file / TS writes performed since the last `SYNCPOINT` (or since the task began) are made permanent; they can no longer be rolled back. A fresh, empty journal is started and subsequent mutations accumulate against it.

`SYNCPOINT` is implicit on `EXEC CICS RETURN` — programs that run to completion never need to call it explicitly. The explicit form is useful in long-running tasks that want to checkpoint partial progress, or in programs that mix recoverable phases with phases where rollback is no longer meaningful.

Options None.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Journal cleared.

`SYNCPOINT` cannot fail in bricks because the underlying bbolt writes have already committed by the time the journal entry was appended.

Example

```
*  Phase 1: build the audit row, commit immediately so a later
*  validation failure does not undo the audit trail.
EXEC CICS WRITEQ TS QUEUE('AUDIT') FROM(WS-LOG) END-EXEC.
EXEC CICS SYNCPOINT END-EXEC.

*  Phase 2: real business work; rolled back if validation fails.
EXEC CICS REWRITE FILE('CUSTOMERS') FROM(WS-CUST) END-EXEC.
IF WS-INVALID
   EXEC CICS SYNCPOINT ROLLBACK END-EXEC
   EXEC CICS ABEND ABCODE('VAL1') END-EXEC
END-IF.
```

SYNCPOINT ROLLBACK

Discard the current unit of work and begin a new one.

Format

```
EXEC CICS SYNCPOINT ROLLBACK END-EXEC
```

Description Walks the task's undo journal in reverse and applies each inverse operation:

- `WRITE` is undone by deleting the new key.
- `REWRITE` is undone by re-writing the captured pre-image.
- `DELETE` is undone by re-writing the deleted record.
- `WRITEQ TS` (append) is undone by deleting the appended item.
- `WRITEQ TS ... REWRITE` is undone by re-writing the pre-image.
- `DELETEQ TS` is undone by restoring every prior item from a snapshot taken before the delete.

After the walk the journal is cleared and a new unit of work begins.

`ROLLBACK` is implicit when `EXEC CICS ABEND` runs **without** a `HANDLE ABEND` exit. When an exit *does* intercept the abend, the exit code decides whether to call `SYNCPOINT ROLLBACK` (typical) or `SYNCPOINT` (commit partial work and continue).

Options `ROLLBACK` is the only option — it is what distinguishes this form from plain `SYNCPOINT`.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	All journal entries successfully reverted.

Condition	EIBRESP	Cause
ROLLEDBACK	82	One or more inverse ops failed (the underlying bbolt error is logged). The journal is still cleared; some writes may remain partially undone.

Example

```
ADDRESS CICS

EXEC CICS WRITE FILE('ACCT') RIDFLD(DEBIT_ID)  FROM(DEBIT_REC)  END-EXEC
EXEC CICS WRITE FILE('ACCT') RIDFLD(CREDIT_ID) FROM(CREDIT_REC) END-EXEC

IF DEBIT_TOTAL <> CREDIT_TOTAL THEN DO
   EXEC CICS SYNCPOINT ROLLBACK END-EXEC      /* both writes undone */
   EXEC CICS ABEND ABCODE('IMBL') END-EXEC
END

EXEC CICS SYNCPOINT END-EXEC                  /* commit both writes */
```

HANDLE CONDITION

Arm program labels to receive control on named EXEC CICS conditions.

Format

```
EXEC CICS HANDLE CONDITION cond1[(label)] cond2[(label)] ...
END-EXEC
```

Description Builds a per-task map from condition name to label. After every EXEC CICS command, when `EIBRESP ≠ NORMAL`, the dispatcher consults the map:

1. If the *exact* condition name is armed, control branches to the named label.
2. Otherwise, if the catch-all `ERROR` is armed, control branches to that label.
3. Otherwise, the response code is left in `EIBRESP` for the program to test, exactly as if `HANDLE CONDITION` had never been issued.

The `(label)` form arms the trap; the bare condition name (no parens) **disarms** it. Arming a previously-armed condition replaces the old label.

`HANDLE CONDITION` is the legacy CICS error-handling style. Modern programs that use `RESP(rc)` style (or REXX `SIGNAL ON ERROR`) do not need it; the two styles can coexist within the same program because both ultimately read `EIBRESP`.

Options **condN(label)** Arm condN (one of the names in Chapter 12. Response codes, e.g. NOTFND, DUPREC, MAPFAIL, INVREQ, IOERR, LENGERR, QIDERR, ITEMERR, ENDFILE, PGMIDERR, EOC) so it branches to label.

condN *(no parens)* Disarm condN.

ERROR(label) Arm a catch-all that fires for any non-NORMAL response not matched by a more specific arm.

A single HANDLE CONDITION may combine any number of arm and disarm options.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Map updated.
INVREQ	16	Unknown condition name.

Example

```
ADDRESS CICS

EXEC CICS HANDLE CONDITION NOTFND(NOREC)
                           DUPREC(DUP)
                           ERROR(OOPS)
END-EXEC

EXEC CICS READ FILE('CUSTOMERS') INTO(REC) RIDFLD(CKEY) END-EXEC
SAY 'Found:' REC
EXIT

NOREC: SAY 'No customer with key' CKEY; EXIT 4
DUP:   SAY 'Duplicate key on insert';  EXIT 8
OOPS:  SAY 'Other CICS error, RC=' EIBRESP; EXIT 12
```

IGNORE CONDITION

Suppress the HANDLE CONDITION trap for one or more conditions.

Format

```
EXEC CICS IGNORE CONDITION cond1 cond2 ...
END-EXEC
```

Description Marks each named condition as *ignored*: even if HANDLE CONDITION previously armed it, the trap will no longer fire and the program must test EIBRESP itself. IGNORE CONDITION stays in effect until a later HANDLE CONDITION cond(label) re-arms the same condition.

The distinction between *unarmed* (never armed, or disarmed by the bare-name form) and *ignored* matters only when `ERROR(label)` is armed: an unarmed condition still falls through to `ERROR`, but an ignored one does not.

Options Each operand is a bare condition name — no parentheses, no label. Multiple conditions may be combined in a single call.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Map updated.
INVREQ	16	Unknown condition name.

Example

```
EXEC CICS HANDLE CONDITION ERROR(BADIO) END-EXEC.

*  We *expect* NOTFND on this READ - don't trip the ERROR trap.
EXEC CICS IGNORE CONDITION NOTFND END-EXEC.

EXEC CICS READ FILE('CUSTOMERS') INTO(WS-REC) RIDFLD(WS-KEY) END-EXEC.
IF EIBRESP = 13
   PERFORM CREATE-NEW-CUSTOMER
END-IF.
```

HANDLE AID

Arm program labels to receive control on a particular attention key.

Format

```
EXEC CICS HANDLE AID key1[(label)] key2[(label)] ...
END-EXEC
```

Description Builds a per-task map from AID byte to label. After any command that updates `EIBAID` (`SEND MAP`, `RECEIVE MAP`, `RECEIVE`), the dispatcher consults the map and branches if the new `EIBAID` matches an armed key.

The bare key name (no parens) disarms a previously-armed key. `ANYKEY` is a catch-all matched by any AID for which no specific arm exists.

Options **keyN(label)** / **keyN** `keyN` is one of `ENTER`, `CLEAR`, `PA1–PA3`, `PF1–PF24`, or `ANYKEY`. Parenthesised arms the trap; bare disarms it.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Map updated.
INVREQ	16	Unknown AID name.

Example

```
ADDRESS CICS

EXEC CICS HANDLE AID PF3(QUIT) PF12(QUIT) CLEAR(QUIT) END-EXEC

DO FOREVER
   EXEC CICS RECEIVE MAP('CUST1') INTO(SCR.) END-EXEC
   ... process ...
   EXEC CICS SEND MAP('CUST1') FROM(SCR.) ERASE END-EXEC
END

QUIT:
   EXEC CICS RETURN END-EXEC
```

HANDLE ABEND

Arm a program-level abend exit.

Format

```
EXEC CICS HANDLE ABEND { LABEL(label) | PROGRAM(name) | CANCEL | RESET }
END-EXEC
```

Description Registers an exit that runs when the task abends (either via `EXEC CICS ABEND` or via an unhandled runtime error). The exit *replaces* the implicit `SYNCPOINT ROLLBACK` that an uncaught abend would perform — it is now the exit's responsibility to call `SYNCPOINT` or `SYNCPOINT ROLLBACK` explicitly.

`EIBABCODE` is set to the abend code (default AAAA) before the exit receives control.

Options **LABEL(label)** Branch to `label` within the current program when the task abends. This is the form REXX programs almost always use.

PROGRAM(name) XCTL to the named program when the task abends. Currently accepted by the parser; the runtime treats `PROGRAM(name)` as equivalent to `LABEL(name)` (i.e. it branches inside the current program rather than transferring control).

CANCEL Disarm the current exit and restore the previous one (one level of nesting is preserved).

RESET Re-enable an exit previously suspended by `CANCEL`.

Exactly one of `LABEL`, `PROGRAM`, `CANCEL`, `RESET` must be given.

Conditions

Condition	EIBRESP	Cause
NORMAL	0	Exit registered or restored.
INVREQ	16	Conflicting / missing options, or `RESET` with no prior exit.

Example

```
ADDRESS CICS

EXEC CICS HANDLE ABEND LABEL(CLEANUP) END-EXEC

EXEC CICS WRITE FILE('LEDGER') RIDFLD(K) FROM(REC) END-EXEC
EXEC CICS WRITE FILE('AUDIT')  RIDFLD(K) FROM(REC) END-EXEC

/* something later may ABEND */
EXIT

CLEANUP:
   SAY 'Caught abend' EIBABCODE
   EXEC CICS SYNCPOINT ROLLBACK END-EXEC      /* undo both writes */
   EXEC CICS RETURN END-EXEC
```

Chapter 11. The Execute Interface Block (EIB)

The EIB is a per-task scratch area populated by the dispatcher before the program runs and by each `EXEC CICS` command on return. In bricks the EIB is exposed as a set of well-known variable names in the program's frame; both REXX and COBOL auto-inject the names that aren't already declared.

Field	Set by	Meaning
`EIBAID`	`SEND MAP` / `RECEIVE MAP`	Single-byte AID character of the most recent map response. Use `C2X(EIBAID)` (REXX) or compare with `X'F3'` (COBOL) to detect PF/PA/CLEAR/ENTER.
`EIBCPOSN`	`SEND MAP` / `RECEIVE MAP`	1-based cursor position of the most recent map response.

Field	Set by	Meaning
EIBCALEN	dispatcher (per-task entry)	Length of DFHCOMMAREA flowed in from the prior task or LINK caller. Zero when no COMMAREA was passed.
EIBTRMID	dispatcher	This terminal's id. Same as ASSIGN TERMID(...).
EIBRESP	every EXEC CICS	The response code of the most recent command (see Chapter 12).
EIBRESP2	every EXEC CICS	The secondary response code of the most recent command (currently always 0).
RC	every EXEC CICS	Mirror of EIBRESP, for REXX IF RC <> 0 style.
DFHCOMMAREA	dispatcher / LINK / RETURN	The COMMAREA bytes; in COBOL, auto-injected as PIC X(2000) if not declared.

REXX

EIBAID and friends are ordinary REXX variables that the handlers write through the `cics.Frame` interface. A REXX program tests them exactly like any other variable:

```
IF EIBRESP <> 0 THEN SAY 'Command failed, RC=' EIBRESP
IF C2X(EIBAID) = 'F3' THEN EXEC CICS RETURN END-EXEC
```

SIGNAL ON ERROR (Chapter 18) is the alternative to per-call IF EIBRESP <>.

COBOL

The same names are auto-injected as PIC items if the program doesn't declare them (`cobol.ensureSystemItems`). Test them with ordinary IF clauses:

```
IF EIBRESP NOT = 0
   DISPLAY 'Command failed'
END-IF.

IF EIBAID = X'F3'
   EXEC CICS RETURN END-EXEC
END-IF.
```

There is no COBOL equivalent of SIGNAL ON ERROR yet.

Chapter 12. Response codes

After every `EXEC CICS` command the handler writes `EIBRESP`, `EIBRESP2`, and `RC` into the program's frame. The full set of constants is in `cics/resp.go`; the values a typical bricks program tests for are:

Constant	Value	Meaning
`NORMAL`	0	Success.
`ERROR`	1	Generic error.
`EOC`	6	End-of-chain (second `RECEIVE` in same task).
`NOTFND`	13	Record / key not found.
`DUPREC`	14	`WRITE` collided with an existing key.
`DUPKEY`	15	Duplicate key on a non-unique index (reserved).
`INVREQ`	16	Invalid request: bad option combination, missing data store, invalid name.
`IOERR`	17	Underlying store / network IO failed.
`NOSPACE`	18	Out of storage (reserved).
`NOTOPEN`	19	File not open (reserved).
`ENDFILE`	20	Browse walked past the last key (or out of `GENERIC` prefix).
`LENGERR`	22	Length mismatch.
`QZERO`	23	Queue is empty (reserved).
`ITEMERR`	26	TS item out of range; typical at end-of-queue.
`PGMIDERR`	27	`LINK` / `XCTL` target not in `transactions.conf`, or sub-program errored, or caller is not authorised for the target.
`MAPFAIL`	36	`SEND MAP` / `RECEIVE MAP` could not find or render the named map.
`QIDERR`	44	TS queue id invalid or unknown.

Idiomatic error handling

REXX, per-call test:

```
EXEC CICS READ FILE('CUSTOMERS') INTO(REC) RIDFLD(K) END-EXEC
SELECT
```

```
   WHEN EIBRESP = 0  THEN NOP
   WHEN EIBRESP = 13 THEN MSG = 'Customer' K 'not found'
   OTHERWISE              MSG = 'I/O error, RC=' EIBRESP
END
```

REXX, signal-on:

```
SIGNAL ON ERROR NAME CICSERR
EXEC CICS READ FILE('CUSTOMERS') INTO(REC) RIDFLD(K) END-EXEC
...
EXIT
CICSERR:
   SAY 'CICS error at line' SIGL ', RC=' RC
   EXIT
```

COBOL:

```
EXEC CICS READ FILE('CUSTOMERS') INTO(REC) RIDFLD(CKEY) END-EXEC
EVALUATE EIBRESP
   WHEN 0   CONTINUE
   WHEN 13  MOVE 'Customer not found' TO MSG
   WHEN OTHER MOVE 'I/O error' TO MSG
END-EVALUATE.
```

Chapter 13. Commands not implemented

The following commands are not implemented and the parser does not recognize them at all:

Command	Use
`GETMAIN`, `FREEMAIN`	Dynamic storage. REXX has dynamic variables; COBOL has `WORKING-STORAGE`.
`ENQ`, `DEQ`	User-level resource locking. File-level locking already exists via `READ … UPDATE`.

Recently added

- **`CONVERSE`** — see Chapter 4. Syntactic sugar for `SEND MAP` + `RECEIVE MAP`.
- **`START` / `RETRIEVE`** — see Chapter 5. Same-terminal scheduling with payload pass-through. Cross-terminal and no-`TERMID` (headless) STARTs are still deferred; the v1 surface rejects them with `RESP-INVREQ` and a clear message.

Part 3. The REXX language

Chapter 14. REXX program structure

A bricks REXX program is a flat sequence of statements with optional labels and procedures. Execution begins at the first statement; the program ends when execution falls off the bottom, or hits EXIT.

```
/* HELO — minimum REXX program */
ADDRESS CICS
EXEC CICS SEND MAP('HELO1') ERASE END-EXEC
EXEC CICS RETURN END-EXEC
```

Procedures

A label followed by `PROCEDURE [EXPOSE list]` defines a procedure with its own variable scope. `EXPOSE` re-routes named variables to the caller's frame recursively.

```
CALL GREET 'Alice'
EXIT

GREET: PROCEDURE EXPOSE LANG.
   PARSE ARG NAME
   SAY LANG.HELLO NAME
   RETURN
```

ADDRESS

`ADDRESS` <env> switches the active command handler. Bare strings inside an `ADDRESS` scope are commands routed to that handler; bricks ships a `CICS` handler.

```
ADDRESS CICS
"SEND MAP('CUST1') FROM(SCR.) ERASE"
"RETURN"
```

Chapter 15. Variables and stems

- **Simple variables** are case-insensitive. An unset variable resolves to its uppercased name (REXX NOVALUE convention).
- **Stems** have a default value plus per-tail values:

  ```
  STEM. = 'unset'
  STEM.42 = 'forty-two'
  SAY STEM.1   /* unset      */
  SAY STEM.42  /* forty-two  */
  ```

- **Compound-variable tail substitution.** Non-numeric tail symbols are resolved at every reference. With `J = 3`, the symbol `A.J` reads or writes `A.3`. Pure numeric tails (`A.0`, `A.42`) and unset tail symbols (REXX NOVALUE) remain literal. Multi-segment tails work too: with `I=1, J=2`, `A.I.J` references `A.1.2`.

- **`DROP name [name…]`** removes one or more variables. A trailing `.` drops the entire stem (default + every tail) — useful for resetting an accumulator between paginated reads: `DROP RECS.`.

See Appendix B for the canonical compound-symbol pitfall.

Chapter 16. Control flow

Construct	Forms
`IF expr THEN [ELSE]`	one-line or block
`SELECT … WHEN … OTHERWISE … END`	`OTHERWISE NOP` works for the empty branch
`DO`, `DO N`, `DO var=a TO b BY s`, `DO WHILE`, `DO UNTIL`, `DO FOREVER`	the usual loop family
`DO var OVER stem.`	iterate over each tail of a stem; numeric tails sort first, then lexicographic
`LEAVE [ctrlvar]`	exit the innermost (or named outer) DO
`ITERATE [ctrlvar]`	skip to the next iteration of the innermost (or named outer) DO
`CALL`, `RETURN`, `EXIT`	procedure call, return, exit
`SIGNAL <label>`	non-local jump
`INTERPRET expr`	evaluate the string value of `expr` as REXX source and execute
`NUMERIC DIGITS n` / `FUZZ n` / `FORM SCIENTIFIC|ENGINEERING`	basic settings honoured; arithmetic is float64 internally
`NOP`	a real no-op statement

Chapter 17. PARSE templates

```
PARSE [UPPER] {VAR var | VALUE … WITH | ARG | PULL} template
```

Template features supported:

- String anchors (`'literal'`).
- Absolute column markers (n).
- Relative column markers (+n / −n).
- The `.` placeholder (skip a token).
- Bare variable runs.

```
PARSE VAR LINE  TID  CKEY  .              /* whitespace tokens */
PARSE VAR REC   NM '|' AD '|' CY '|' PH   /* '|' delimiter     */
PARSE VAR ROW   1 NAME 21 ADDR 51 PHONE   /* fixed columns     */
```

Chapter 18. Conditions and SIGNAL ON

`SIGNAL ON {ERROR | NOVALUE | SYNTAX | HALT} [NAME label]` arms a condition. When it fires:

- `SIGL` is set to the source line of the failing statement.
- Control jumps to the labelled handler.

`SIGNAL OFF cond` disarms.

Condition	Fires on
`ERROR`	An `EXEC CICS` command returns a non-zero RC.
`NOVALUE`	A reference to an unset simple or compound variable.
`SYNTAX`	Any other interpreter error — bad numeric, divide by zero, unknown function, etc.
`HALT`	An external halt request.

Example

```
SIGNAL ON ERROR  NAME CICSERR
SIGNAL ON SYNTAX NAME OOPS

ADDRESS CICS
EXEC CICS READ FILE('CUSTOMERS') INTO(REC) RIDFLD(CKEY) END-EXEC
EXIT

CICSERR:
   SAY 'EXEC CICS failed at line' SIGL ', RC=' RC
   EXIT 12

OOPS:
   SAY 'Interpreter error at line' SIGL
   EXIT 16
```

Without an armed trap, the legacy "test EIBRESP after every verb" pattern still works. For per-condition (rather than blanket) traps, use `EXEC CICS HANDLE CONDITION` (Chapter 10); the two styles can coexist.

Chapter 19. Built-in functions

Family	Functions
Length / index	`LENGTH`, `POS`, `LASTPOS`, `WORDS`, `WORDPOS`, `WORDINDEX`, `WORDLENGTH`, `COUNTSTR`

Family	Functions
Substring	SUBSTR, LEFT, RIGHT, SUBWORD, WORD, DELSTR, DELWORD, INSERT, OVERLAY, CHANGESTR
Whitespace / case	STRIP, SPACE, CENTER (alias CENTRE), JUSTIFY, UPPER, LOWER, REVERSE, COPIES, ABBREV
Translation	TRANSLATE, VERIFY, COMPARE, XRANGE, BITAND, BITOR, BITXOR
Conversion	C2X, X2C, D2X, X2D, D2C, C2D, B2X, X2B
Numeric	ABS, MAX, MIN, INT, TRUNC, MOD (= // operator), SIGN, FORMAT(num, before, after), DIGITS, FUZZ, FORM
Type / data	DATATYPE (with N, W, A options), LENGTH
Date / time	DATE (N/S/E/U/O/B; B does basedate arithmetic), TIME
Stream I/O	LINEIN, LINEOUT, LINES, CHARIN, CHAROUT, CHARS, STREAM
Variables / args	VALUE, ARG, RANDOM, ERRORTEXT

`VALUE` accepts the canonical 1-arg read form (`VALUE('SCR.ROW' || J)`) and the 2-arg assignment form (`CALL VALUE 'SCR.ROW' || J, LINE` — sets the variable named at runtime and returns the prior value).

`C2X` is the standard way to compare `EIBAID` to a PF-key code: `IF C2X(EIBAID) = 'F7' THEN` … for PF7.

`DATE('B')` and `DATE('B', 'YYYYMMDD', 'S')` give days since 0001-01-01 — subtract two basedates for an exact day delta.

Stream I/O

The stream functions read and write text files in the `tmp_dir` sandbox — the same backend that powers COBOL's `EXEC CICS READQ TD` / `WRITEQ TD` / `DELETEQ TD` (see Chapter 9). The sandbox is strict: ASCII only, LF-terminated, no traversal.

Function	Form	Behaviour
`LINEIN(name)`	next line	Auto-opens `name` for read on first call; returns the next line (LF stripped). Empty string at EOF.
`LINEIN(name, 1)`	rewind	Closes and reopens `name`, then returns line 1. Only 1 is honoured; any other line number is treated as the "next line" form.

Function	Form	Behaviour
`LINEOUT(name)`	close	Closes the named stream. Returns `0`.
`LINEOUT(name, line)`	append	Auto-opens `name` for append on first call; appends `line || '\n'`. Returns `0` on success, 1 on error.
`LINES(name)`	EOF probe	Returns 1 while more data is available, `0` at EOF.
`CHARIN(name, start, n)`	byte read	Reads n bytes (default 1). `start = 1` rewinds first; other `start` values are ignored.
`CHAROUT(name, s)`	byte write	Appends raw bytes. Same ASCII/LF validation as `LINEOUT`.
`CHARS(name)`	bytes remaining	Returns the file size minus the current read position.
`STREAM(name, 'S')`	state	Returns `'READY'`, `'NOTREADY'`, or `'ERROR'`.
`STREAM(name, 'D')`	description	Returns a short status string (`'OK'` / parse-time reason).
`STREAM(name, 'C', cmd)`	command	`OPEN READ` / `OPEN WRITE` / `OPEN APPEND` / `CLOSE` / `QUERY EXISTS` / `QUERY SIZE` / `DELETE`.

```
/* Append a row, then read every row back. */
CALL LINEOUT 'export.txt', 'C0000099|WIDGET-Z|7|9.95'
CALL LINEOUT 'export.txt'                    /* close the writer    */
DO WHILE LINES('export.txt') > 0
   SAY LINEIN('export.txt')
END
CALL STREAM 'export.txt', 'C', 'CLOSE'
```

Every handle a program opens is closed automatically at task end; an interpreter that forgets to call `CLOSE` does not leak descriptors. A sandbox violation (`'../etc/passwd'`, `'sub/x'`, leading-dot) causes the next `LINEIN` / `LINEOUT` to raise an error and `STREAM('S')` to report `'ERROR'`.

Operators

`+ - * / % // **`, comparisons (numeric when both sides parse as numbers, trimmed-string otherwise), `||` and juxtaposition concat, `&` `|`, unary `\`.

Part 4. The COBOL language

Chapter 20. COBOL source format

Bricks ships a free-form COBOL interpreter (`cobol/`) that sits beside REXX as a second front end on the same `EXEC CICS` surface. The language-specific layer is `cobol/frame.go`, which adapts COBOL's group-item world to the REXX-style `STEM.TAIL` lookup the CICS handlers use.

Source rules

- **Free-form.** No column 1-72 ruling; no Area A / Area B.
- **Hyphens** are allowed in identifiers (`CUST-RECORD`).
- **Comments** use modern *> anywhere on a line, or legacy * in column 1.
- **Strings** use `'...'` or `"..."` with quote-doubling for embedded quotes.
- **Hex literals** like `X'F3'` and `X"7C"` decode to their byte value — used for `IF EIBAID = X'F3'` to check PF3 / PF12 / etc. without `C2X(EIBAID) = 'F3'` round-trips.

Divisions

The four divisions parse in their canonical order:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. HELC.

ENVIRONMENT DIVISION.        *> optional, must be empty

DATA DIVISION.
WORKING-STORAGE SECTION.
01 SCR.
   05 INFOLINE PIC X(78).
   05 GREETING PIC X(20).

PROCEDURE DIVISION.
MAIN.
    MOVE 'Hello' TO GREETING.
    EXEC CICS SEND MAP('HELO1') FROM(SCR) ERASE END-EXEC.
    EXEC CICS RETURN END-EXEC.
    STOP RUN.
```

`LINKAGE SECTION` is not yet parsed; `DFHCOMMAREA` is auto-injected as `PIC X(2000)` if the program doesn't declare it (see `cobol.ensureSystemItems`), so a sub-program can `MOVE DFHCOMMAREA TO key` immediately. The dispatcher strips trailing space when reading the COBOL frame's `DFHCOMMAREA` back out, so a fixed-width buffer round-trips cleanly across an inter-language `EXEC CICS LINK`.

Chapter 21. DATA DIVISION

- **PIC clauses:** `X(n)` alphanumeric, `9(n)` integer, `S9(n)` signed, `9(n)V99` decimal (the `V` is positional, no real binary scaling yet — arithmetic is float64 internally).
- **VALUE:** `VALUE 'literal'`, `VALUE 42`, `VALUE SPACES`, `VALUE ZEROS`, `VALUE HIGH-VALUES`, `VALUE LOW-VALUES`, `VALUE QUOTES`.
- **Group items:**

  ```
  01 PARENT.
     05 CHILD PIC X(8).
     05 OTHER PIC X(4).
  ```

  Children are stored as offsets into a single parent buffer, so `MOVE` to the parent fans out and `EXEC CICS SEND MAP FROM(PARENT)` walks the children for field values.

- **Data names can be reused across groups.** A child name that appears under more than one group is fine; the parser accepts the declaration and tracks the collision in `Program.AmbiguousNames`. Unqualified access to such a name is a runtime error ("ambiguous reference") — disambiguate with `OF` (or its synonym `IN`):

  ```
  01 SCR.
     05 CUSTNO PIC X(8).
  01 DET.
     05 CUSTNO PIC X(8).
  ...
  MOVE 'A1234567' TO CUSTNO OF SCR.
  MOVE 'B7654321' TO CUSTNO OF DET.
  DISPLAY CUSTNO OF SCR.
  ```

  Single-step qualification (`X OF Y`) picks the matching child anywhere in Y's subtree; multi-step (`X OF Y OF Z`) chains through nested groups. Bare names that are unique across the program continue to work without qualification — the pre-7 `runtime/cobol/gust.cob` convention of prefixed child names (`DCUSTNO`, `DNAME`, `DMSG`) still compiles and runs, it just isn't required any more.

  A sibling-duplicate (two children of the SAME parent sharing a name) is still a parse-time error — no qualifier can disambiguate between siblings of one group.

Chapter 22. PROCEDURE DIVISION

Statements supported

`MOVE`, `DISPLAY`, `STOP RUN`, `GOBACK`, `EXIT`, `EXIT PROGRAM`, `CONTINUE` (no-op), `IF ... [ELSE] ... END-IF`, `EVALUATE subject WHEN value [WHEN value] ... [WHEN OTHER] ... END-EVALUATE`, `PERFORM para`, `PERFORM para UNTIL cond`, `PERFORM para N TIMES`, `PERFORM para VARYING idx FROM x BY y UNTIL cond`, `GO TO para`, `COMPUTE target [ROUNDED] = expr [ON SIZE ERROR ... END-COMPUTE]`, `ADD a TO b [GIVING c] [ROUNDED] [ON SIZE ERROR ... END-ADD]`, `SUBTRACT a FROM b [GIVING c] [ROUNDED] [ON SIZE ERROR ...`

END-SUBTRACT], MULTIPLY a BY b [GIVING c] [ROUNDED] [ON SIZE ERROR ... END-MULTIPLY], DIVIDE a INTO b [GIVING c] [ROUNDED] [ON SIZE ERROR ... END-DIVIDE], DIVIDE a BY b GIVING c [ROUNDED] [ON SIZE ERROR ... END-DIVIDE], STRING ... DELIMITED BY (SIZE | 'lit') INTO target END-STRING, UNSTRING source DELIMITED BY 'lit' INTO t1 t2 ... END-UNSTRING, INSPECT subject TALLYING counter FOR (ALL | LEADING | CHARACTERS) [needle] [BEFORE/AFTER INITIAL delim], INSPECT subject REPLACING (ALL | LEADING | FIRST | CHARACTERS) [needle] BY replacement [BEFORE/AFTER INITIAL delim], EXEC CICS ... END-EXEC.

Every target name above (the second operand of MOVE, the target of COMPUTE / ADD / SUBTRACT / MULTIPLY / DIVIDE / GIVING / STRING INTO / UNSTRING INTO / INSPECT, and the subject of EVALUATE / IF) accepts qualification via OF / IN (see the Data Division section on globally non-unique names) and subscripts via (idx) for OCCURS-resident items.

ROUNDED and ON SIZE ERROR

ROUNDED applied to COMPUTE and the four arithmetic verbs rounds the final value half-away-from-zero to the destination's PIC scale instead of truncating toward zero (the unROUNDED default).

ON SIZE ERROR runs its statement list when the calculation overflows the int64 fixed-point engine, the result's integer-part digits don't fit in the target PIC, or a DIVIDE divisor is zero; when present, the destination is left untouched on a size-error firing. Without an ON SIZE ERROR branch the value is silently truncated/wrapped as before so existing programs aren't disrupted.

PERFORM N TIMES and PERFORM VARYING

```
PERFORM FILL-ROW 15 TIMES.

PERFORM FILL-ROW
    VARYING I FROM 1 BY 1 UNTIL I > 15.
```

PERFORM ... N TIMES runs the named paragraph exactly N times; N can be a numeric literal or a numeric data item. PERFORM ... VARYING idx FROM x BY y UNTIL cond initialises idx to x, runs the body while cond is false, then increments idx by y between iterations.

OCCURS arrays

```
01 TABLE-AREA.
   05 ROW OCCURS 15 TIMES.
      10 K  PIC X(8).
      10 N  PIC X(28).
01 I  PIC 9(4).
...
MOVE 'KEY-A' TO K(1).
MOVE 'KEY-B' TO K(I).
PERFORM SHOW-ROW VARYING I FROM 1 BY 1 UNTIL I > 15.
```

OCCURS n TIMES declares n copies of an item; references add a parenthesised 1-based subscript expression that picks one. The subscript can be any numeric expression — a literal, a data item, or arithmetic — and is bounds-checked at runtime. Subscripts work on group items (ROW(5)) and on elementaries inside the OCCURS group (`K(5)`) alike. The cap is 4096 occurrences per item; nested OCCURS (an OCCURS group inside another OCCURS group) is rejected at parse time.

Periods

Periods are scope-terminators, not per-statement: a statement inside an `IF ... END-IF` body has no period; only the unscoped statement at the paragraph level does.

Postfix NOT

IBM-style postfix NOT (`X NOT = Y`, `X NOT > Y`) is supported in `IF` / `EVALUATE` / `PERFORM UNTIL` conditions.

Intrinsic functions

The shipped intrinsic library is:

Function	Args	Result
`FUNCTION UPPER-CASE(s)`	1 alphanumeric	s uppercased.
`FUNCTION LOWER-CASE(s)`	1 alphanumeric	s lowercased.
`FUNCTION TRIM(s)`	1 alphanumeric	s with leading and trailing spaces removed (both ends; one-arg form).
`FUNCTION REVERSE(s)`	1 alphanumeric	s reversed (rune-aware).
`FUNCTION LENGTH(s)`	1 alphanumeric	numeric byte length of the operand's storage (PIC X(8) □ 8).
`FUNCTION NUMVAL(s)`	1 alphanumeric	numeric value of s after stripping leading / trailing whitespace; errors on non-numeric content (does not silently coerce to zero).
`FUNCTION POS(needle, haystack)`	2 alphanumeric	1-based byte position of needle in haystack, or 0 when not found. Mirrors REXX's `POS()`. Empty needle returns 0.

Numeric-returning intrinsics (`LENGTH`, `NUMVAL`, `POS`) participate in `IF` / `EVALUATE` / arithmetic comparisons as numeric operands, so `IF FUNCTION POS(NEEDLE, HAY) > 0` works without an intermediate `COMPUTE`. The remaining intrinsics return alphanumeric and compare by bytes (rstripped).

`FUNCTION POS` is the substring-search hook used by `runtime/cobol/gusl.cob` to filter the customers file when the operator types a search term at GUST's `S` action.

INSPECT

```
INSPECT subject TALLYING counter FOR (ALL | LEADING | CHARACTERS)
                                     [needle] [BEFORE/AFTER INITIAL delim]
                                     [, ...]
INSPECT subject REPLACING (ALL | LEADING | FIRST | CHARACTERS)
                          [needle] BY replacement
                          [BEFORE/AFTER INITIAL delim]
                          [, ...]
```

Quantifiers:

- **ALL** — every occurrence of `needle` (TALLYING) or every occurrence rewritten (REPLACING).
- **LEADING** — only successive occurrences at the start of the in-scope region.
- **FIRST** — only the first occurrence (REPLACING only).
- **CHARACTERS** — every character in the in-scope region; for TALLYING this just counts characters, for REPLACING it overwrites each one with the first byte of `replacement`.

Limiters:

- **BEFORE INITIAL delim** narrows the in-scope region to everything before the first occurrence of `delim`.
- **AFTER INITIAL delim** narrows to everything after the first occurrence of `delim`.

Both can be set on the same phrase; AFTER picks the start, BEFORE picks the end.

Multiple phrases can be combined per statement, optionally separated by commas:

```
INSPECT REC REPLACING ALL ',' BY '|' ALL ';' BY '/'.
INSPECT REC TALLYING C FOR LEADING '0', ALL 'X' AFTER INITIAL ' '.
```

Counter behaviour: TALLYING **accumulates** — successive `INSPECT TALLYING` statements add to the existing counter value rather than resetting, matching IBM semantics. REPLACING phrases run left-to- right in declared order, so later phrases see the output of earlier ones.

Subject, counter, and operands all accept qualified data-names (`INSPECT REC OF DET ...`). Trailing PIC X padding is rstripped from both the subject and the needle before matching, so a 30-byte filter containing `FOO` plus trailing spaces matches against the meaningful content of the subject rather than against the padding.

IBM's CONVERTING form is not parsed yet.

EVALUATE limitations

`EVALUATE` only supports the simple value form. `EVALUATE TRUE` / `EVALUATE FALSE` is rejected at parse time with a hint to rewrite as `IF/ELSE` — those forms (and `WHEN ... THRU ...` ranges, multi-subject `ALSO` clauses, condition-name arms) are deferred.

Chapter 23. The EIB block in COBOL

`EIBRESP`, `EIBRESP2`, `EIBAID`, `EIBCPOSN`, `EIBCALEN`, `EIBTRMID`, `RC`, and `DFHCOMMAREA` are auto-injected if not declared, so most programs need no boilerplate. After every `EXEC CICS`, `EIBRESP` and `EIBRESP2` are populated by the handler the same way they are for REXX. The legacy "test `EIBRESP` after every verb" pattern is the most common idiom; for non-local error handling without a REXX-style `SIGNAL ON ERROR`, use `EXEC CICS HANDLE CONDITION` / `EXEC CICS HANDLE ABEND` (Chapter 10).

See Chapter 11 for the field meanings.

Chapter 24. EXEC CICS in COBOL

The COBOL parser collects every token between `EXEC CICS` and `END-EXEC` and reconstructs the body verbatim for the same `cics.ParseCommand` REXX uses. Three consequences:

- **Map-field names must match group-child names exactly.** When `EXEC CICS SEND MAP('CUST1') FROM(SCR)` fires, the CICS handler asks the frame for `SCR.INFOLINE`, `SCR.MSG`, etc. — so SCR must have children spelled exactly as the map declares fields. Real COBOL solves this with BMS-generated copybooks; bricks doesn't ship those, so the operator declares fields by hand.
- **`DFHCOMMAREA` is the COMMAREA marshalling slot.** Caller passes bytes in via the frame; sub-program reads with `MOVE DFHCOMMAREA TO ...`, mutates a working copy, and `MOVE ... TO DFHCOMMAREA`. Trailing space is stripped by the dispatcher on the way back out.
- **Command-line arguments arrive via `EXEC CICS RECEIVE INTO(...)`.** When the operator types `EXAM 1 2 3` at the blank prompt, the dispatcher hands the unedited line (TRANSID prefix included) to the first task in the chain. The program reads it with `EXEC CICS RECEIVE INTO(WS-INPUT) LENGTH(WS-LEN) END-EXEC` and parses with `UNSTRING WS-INPUT DELIMITED BY ' ' INTO TRANSID A B C END-UNSTRING`. Single-shot per task, then `EOC`. See `runtime/cobol/exam.cob`.

`EXEC CICS SEND TEXT FROM(area) [ERASE]` works the same way. Reconstruction routes through the shared `cics.Handler`, so any verb REXX can issue, COBOL can issue too. The body is laid out as a flat row-major buffer (every `cols` bytes = one row, default 80) — 3270 has no LF, so build `area` as a group of `PIC X(80)` children and fill them row by row, or compose a single `PIC X(n*80)` with a matching `MOVE` per row.

The full `EXEC CICS` verb set is documented in Part 2; behaviour is identical between REXX and COBOL.

Chapter 25. Copybooks

A copybook is a fragment of COBOL source — typically declarations for constants and groups — kept in a separate file and pulled into a program at parse time with a single line:

```
WORKING-STORAGE SECTION.
COPY DFHAID.
COPY DFHRESP.
01 OWN-COUNTER PIC 9(4).
```

bricks expands every `COPY name.` directive at the source-text level before the lexer runs, so the rest of the parser sees one unbroken stream of COBOL.

The COPY directive

Syntax:

```
COPY <name>.
```

- The directive must be on its own line. Anything before or after the `COPY` keyword on the same line is a parse error.
- `<name>` is the basename of a copybook file; leave the extension off. The lookup tries `<name>.cpy`, then `<name>.cbl`, then the exact `<name>`, all case-insensitive.
- The trailing period is required.
- Leading whitespace is tolerated, and an `*>` inline comment on the same line is fine: `COPY DFHAID. *> 3270 AID bytes`.
- The directive is case-insensitive (`COPY`, `Copy`, `copy` all work), and so is the name (`COPY DFHAID.` and `copy dfhaid.` resolve to the same file).

Where bricks searches

The search directory is configured by `copybook_dir=` in `bricks.cnf`. The default is `runtime/cobolcopy/` (auto-derived from `runtime_dir` when that line is set). The standard CICS copybooks ship under this directory, so a fresh install already has `DFHAID.cpy` and `DFHRESP.cpy` available.

Copybook names are restricted to a flat alphanumeric namespace: `A-Z`, `a-z`, `0-9`, `-`, `_`, and `.`. Names containing a path separator, a `..` sequence, or a leading dot are rejected before any filesystem access. A copybook itself may contain `COPY` directives; nesting is capped at depth 5 and cycles are detected explicitly.

Delivered copybook: DFHAID

`COPY DFHAID.` brings in the standard 3270 attention-identifier (AID) byte constants. Every AID is declared **twice**, once under the friendly bricks mnemonic and once under the IBM-traditional `DFH` alias — both names resolve to the same byte, and a program may use either style or mix them.

bricks mnemonic	IBM alias	AID byte	Key
ENTER	DFHENTER	X'7D'	Enter
CLEAR	DFHCLEAR	X'6D'	Clear
PA1	DFHPA1	X'6C'	PA1
PA2	DFHPA2	X'6E'	PA2
PA3	DFHPA3	X'6B'	PA3
PF01	DFHPF1	X'F1'	PF1

bricks mnemonic	IBM alias	AID byte	Key
PF02	DFHPF2	X'F2'	PF2
PF03	DFHPF3	X'F3'	PF3
PF04	DFHPF4	X'F4'	PF4
PF05	DFHPF5	X'F5'	PF5
PF06	DFHPF6	X'F6'	PF6
PF07	DFHPF7	X'F7'	PF7
PF08	DFHPF8	X'F8'	PF8
PF09	DFHPF9	X'F9'	PF9
PF10	DFHPF10	X'7A'	PF10
PF11	DFHPF11	X'7B'	PF11
PF12	DFHPF12	X'7C'	PF12
PF13	DFHPF13	X'C1'	PF13
PF14	DFHPF14	X'C2'	PF14
PF15	DFHPF15	X'C3'	PF15
PF16	DFHPF16	X'C4'	PF16
PF17	DFHPF17	X'C5'	PF17
PF18	DFHPF18	X'C6'	PF18
PF19	DFHPF19	X'C7'	PF19
PF20	DFHPF20	X'C8'	PF20
PF21	DFHPF21	X'C9'	PF21
PF22	DFHPF22	X'4A'	PF22
PF23	DFHPF23	X'4B'	PF23
PF24	DFHPF24	X'4C'	PF24

The bricks mnemonic uses uniform-width `PF01`..`PF24`; the IBM alias keeps the no-leading-zero form (`DFHPF1`..`DFHPF24`) everyone recognises from real CICS code.

Delivered copybook: DFHRESP

`COPY DFHRESP.` brings in the EXEC CICS condition-code constants for `EIBRESP`. Each code is declared twice as well — `RESP-X` mnemonic and `DFHRESP-X` traditional alias. Numeric values match the runtime's emitter table (`cics/resp.go`), so any code bricks returns from a verb has a named constant here.

The most useful entries:

bricks mnemonic	IBM alias	EIBRESP	Condition
RESP-NORMAL	DFHRESP-NORMAL	0	success
RESP-ERROR	DFHRESP-ERROR	1	generic error
RESP-EOC	DFHRESP-EOC	6	end-of-chain / nothing to RECEIVE
RESP-NOTFND	DFHRESP-NOTFND	13	record / item / file not found
RESP-DUPREC	DFHRESP-DUPREC	14	duplicate record on WRITE

bricks mnemonic	IBM alias	EIBRESP	Condition
`RESP-DUPKEY`	`DFHRESP-DUPKEY`	15	duplicate key in browse
`RESP-INVREQ`	`DFHRESP-INVREQ`	16	invalid request (bad args, sandbox violation)
`RESP-IOERR`	`DFHRESP-IOERR`	17	underlying I/O failure
`RESP-NOSPACE`	`DFHRESP-NOSPACE`	18	TS / TD queue full
`RESP-NOTOPEN`	`DFHRESP-NOTOPEN`	19	file not open / dataset not enabled
`RESP-ENDFILE`	`DFHRESP-ENDFILE`	20	end of browse
`RESP-LENGERR`	`DFHRESP-LENGERR`	22	length mismatch
`RESP-QZERO`	`DFHRESP-QZERO`	23	TS / TD queue empty (READQ TD returns this)
`RESP-ITEMERR`	`DFHRESP-ITEMERR`	26	TS item number out of range
`RESP-PGMIDERR`	`DFHRESP-PGMIDERR`	27	LINK / XCTL program not found
`RESP-MAPFAIL`	`DFHRESP-MAPFAIL`	36	RECEIVE MAP with no input
`RESP-INVMPSZ`	`DFHRESP-INVMPSZ`	38	map size mismatch
`RESP-QIDERR`	`DFHRESP-QIDERR`	44	TS queue id unknown
`RESP-ROLLEDBACK`	`DFHRESP-ROLLEDBACK`	82	unit-of-work rolled back

The full list (every code bricks emits) is in `runtime/cobolcopy/DFHRESP.cpy`. Open the file to see the codes not summarised above (`RESP-ILLOGIC`, `RESP-SIGNAL`, `RESP-QBUSY`, the various `RESP-INV...` codes, `RESP-SYSIDERR`, etc.).

Delivered copybook: SQLCA

`COPY SQLCA.` brings in named constants for `SQLCODE` and the most common `SQLSTATE` values, so embedded-SQL programs can write:

```
COPY SQLCA.
...
EXEC SQL SELECT name INTO :NM
         FROM customers WHERE id = :CUSTID END-EXEC.
EVALUATE SQLCODE
    WHEN SQL-OK            PERFORM SHOW-ROW
    WHEN SQL-NODATA        PERFORM SHOW-NOT-FOUND
    WHEN SQL-MULTIPLEROWS  PERFORM SHOW-AMBIGUOUS
    WHEN OTHER             PERFORM SHOW-SQL-ERROR
END-EVALUATE.
```

The SQLCA fields themselves (`SQLCODE`, `SQLSTATE`, `SQLERRMC`) are auto-injected by the bricks

COBOL parser — no `COPY` needed for the data items. SQLCA only adds the constants programs compare against.

bricks mnemonic	IBM alias	SQLCODE	Meaning
`SQL-OK`	`DB2-SUCCESS`	0	success
`SQL-NODATA`	`DB2-NOTFOUND`	+100	no row / end-of-cursor
`SQL-NOTFND`	`DB2-NOTFOUND`	+100	alias
`SQL-NOCONFIG`	—	-1	SQL not configured in bricks.cnf
`SQL-DDLREJECTED`	—	-2	DDL rejected (CEDA-only)
`SQL-GENERIC`	—	-100	generic PG error (see SQLERRMC + log)
`SQL-MULTIPLEROWS`	`DB2-DUPLICATE`	-811	SELECT INTO returned >1 row

SQLSTATE constants are PIC X(5) and compare directly:

```
IF SQLSTATE = SQLSTATE-DUP-KEY THEN
    MOVE 'Already exists.' TO MSG
END-IF.
```

Constant	SQLSTATE	Class
`SQLSTATE-OK`	`00000`	success
`SQLSTATE-NODATA`	`02000`	no-data
`SQLSTATE-WARNING`	`01000`	warning
`SQLSTATE-NOT-NULL`	`23502`	NOT NULL constraint violated
`SQLSTATE-FK-VIOL`	`23503`	foreign-key constraint violated
`SQLSTATE-UQ-VIOL`	`23505`	UNIQUE constraint violated
`SQLSTATE-DUP-KEY`	`23505`	alias
`SQLSTATE-CHK-VIOL`	`23514`	CHECK constraint violated
`SQLSTATE-CONN-FAIL`	`08001`	unable to connect to PG
`SQLSTATE-CONN-LOST`	`08006`	connection dropped mid-flight
`SQLSTATE-INV-AUTH`	`28000`	PG authentication failed
`SQLSTATE-INV-PWD`	`28P01`	wrong password
`SQLSTATE-SYNTAX`	`42601`	SQL syntax error
`SQLSTATE-UNDEF-TBL`	`42P01`	table doesn't exist
`SQLSTATE-UNDEF-COL`	`42703`	column doesn't exist
`SQLSTATE-UNDEF-FN`	`42883`	function doesn't exist
`SQLSTATE-AMBIG-COL`	`42702`	ambiguous column reference
`SQLSTATE-INSUF-PRIV`	`42501`	permission denied

The bricks SQL executor also logs every non-zero `SQLCODE`'s full PG error text via `brickslog.Sys` (one line on the operator console + the per-run log file). Programs render only the short mnemonicdriven message on the 3270 screen; the diagnostic detail lives in the log so PG hostnames / port numbers / wrapper text don't leak to terminal users.

Idiom: replacing raw literals

Before — opaque hex and bare integers, easy to mis-key:

```
IF EIBAID = X'F3' THEN
    MOVE 'Y' TO EXIT-FLAG
END-IF.
IF EIBRESP = 13 THEN
    MOVE 'Customer not found.' TO MSG
END-IF.
IF EIBRESP = 23 THEN          *> TD queue empty
    MOVE 'Y' TO DONE-FLAG
END-IF.
```

After — names a reader can scan:

```
WORKING-STORAGE SECTION.
COPY DFHAID.
COPY DFHRESP.
...
IF EIBAID = PF03 THEN          *> or DFHPF3 if you prefer
    MOVE 'Y' TO EXIT-FLAG
END-IF.
IF EIBRESP = RESP-NOTFND THEN
    MOVE 'Customer not found.' TO MSG
END-IF.
IF EIBRESP = RESP-QZERO THEN
    MOVE 'Y' TO DONE-FLAG
END-IF.
```

Live examples: `qagc.cob` (PF03 cancel branch), `gusl.cob` (EVALUATE EIBAID with PF03 / PF07 / PF08 / PF12), `ordr.cob` (RESP-QZERO and RESP-DUPREC against READQ TD / WRITE FILE), `exam.cob` (RESP-EOC after RECEIVE). All under `runtime/cobol/`.

Restrictions

The bricks COPY preprocessor is deliberately minimal. The following real-CICS extensions are **not** supported:

- `COPY ... REPLACING ==X== BY ==Y==.` — token substitution. Rare in modern code; deferred until a concrete use case appears.
- `COPY xyz OF DFHCOB.` / `COPY xyz IN libname.` — library qualifier. bricks has one search root per process.
- `COPY xyz SUPPRESS.` — suppress copybook listing. bricks produces no listing to suppress.
- Embedded directives: multiple statements on one line including a `COPY` are rejected. The line must be `COPY <name>.` and nothing else.

If a copybook is edited, programs that include it will pick up the new contents only after the program's own source mtime changes (the program cache keys by the program path + mtime, not the copybook's). A trivial way to force a reload is to touch the COBOL file:

```
touch runtime/cobol/qagc.cob
```

Or use `CEMT P C` from a TSO/3270 session to flush the entire program cache.

Chapter 26. Embedded SQL (COBOL)

bricks accepts the classic IBM-style `EXEC SQL … END-EXEC` embedded-SQL surface and runs it against the Postgres connection configured via `db_*` in `bricks.cnf` (see SQL support — Phase 1). Programs read and write rows through prepared statements; the SQLCA fields `SQLCODE`, `SQLSTATE`, `SQLERRMC` are auto-injected into every program's DATA DIVISION so the program can test them without declaring them.

Scope: single-row CRUD (`SELECT INTO`, `INSERT`, `UPDATE`, `DELETE`, `COMMIT`, `ROLLBACK`), the four-verb cursor lifecycle (`DECLARE` / `OPEN` / `FETCH` / `CLOSE`), per-task `CONNECT TO 'name'` database switching, null indicators (`:hostvar :indicator`), and the `WHENEVER` declarative error-handling directive. The same surface works identically in COBOL and REXX.

Format

```
EXEC SQL <statement> END-EXEC.
```

Where `<statement>` is any of the supported verbs (below). Host variables are written with a leading colon: `:CUSTID`, `:NM`, `:BALANCE`. The variable name must already exist in DATA DIVISION — the SQL executor reads its current value (for INPUT bindings) and writes back (for SELECT INTO targets).

Supported verbs

Verb	Notes
`SELECT cols INTO :v, :v FROM …`	Single-row read. Returns +100 (no-data) when 0 rows, -811 when >1 row.
`INSERT INTO t (…) VALUES (…)`	Host-variable references bound as $N placeholders.
`UPDATE t SET … WHERE …`	Returns RESP-NORMAL even when 0 rows match (matches DB2).
`DELETE FROM t WHERE …`	Same row-count semantics as UPDATE.
`COMMIT WORK`	Commits the per-task PG transaction. Also fired by `EXEC CICS SYNCPOINT`.
`ROLLBACK WORK`	Rolls back. Also fired by `EXEC CICS SYNCPOINT ROLLBACK`.

DDL (`CREATE DATABASE`, `DROP DATABASE`, `CREATE USER`, `DROP USER`, `ALTER ROLE`, `CREATE TABLESPACE`, …) is **rejected** with `SQLCODE = -2` and `SQLSTATE = 42501`. CEDA DATABASE owns those operations and audits each one through `brickslog.Audit`. `CREATE TABLE` / `ALTER TABLE` / similar non-privileged DDL is not blocked but is also unsupported in the executor's statement classification — porters should run schema migrations with `psql`.

SQLCODE catalog

bricks adopts the IBM DB2 convention adapted to Postgres. The catalog below covers what programs typically test:

SQLCODE	SQLSTATE	Meaning
`0`	`00000`	Success.
`+100`	`02000`	No data: SELECT INTO returned 0 rows, or an update/delete affected 0 rows.
`-1`	empty	No database configured (no `db_*` lines in `bricks.cnf`).
`-2`	`42501`	DDL rejected (CEDA owns CREATE/DROP DATABASE
`-100`	varies	Generic SQL failure — PG returned an error not specifically mapped.
`-811`	`21000`	SELECT INTO returned more than one row.

`SQLERRMC` always carries the human-readable message from Postgres (truncated to 70 chars to match the PIC X(70) auto- injection). When the error path includes a specific SQLSTATE, that 5-char code lands in `SQLSTATE`; otherwise `SQLSTATE` is the relevant standard code (`00000` on success, `02000` on no-data, `HV000` when the failure didn't carry a state).

SYNCPOINT integration

Each task lazily begins a Postgres transaction on its first `EXEC SQL` that touches data. The transaction commits when:

- `EXEC CICS SYNCPOINT` runs.
- `EXEC SQL COMMIT WORK` runs.
- The task ends cleanly (RETURN, GOBACK, STOP RUN).

And rolls back when:

- `EXEC CICS SYNCPOINT ROLLBACK` runs.
- `EXEC SQL ROLLBACK WORK` runs.
- The task abends via `EXEC CICS ABEND`.

Postgres-side commit fires *before* the bricks-side bbolt journal commit. That ordering avoids a hung PG commit leaving the bricks journal already finalised; failures during the SQL commit surface as ROLLEDBACK from SYNCPOINT, and the bricks journal then rolls back too.

Idiomatic test

```
       EXEC SQL
           SELECT name INTO :NM
           FROM customers_sql
           WHERE id = :CUSTID
       END-EXEC.

       EVALUATE SQLCODE
           WHEN RESP-NORMAL    PERFORM SHOW-NAME       *> +0
           WHEN RESP-NOTFND    PERFORM SHOW-NOTFOUND   *> wait -- that's an EIBRESP cod
           WHEN OTHER          PERFORM SHOW-SQL-ERR
       END-EVALUATE.
```

The RESP-NORMAL constant (from COPY DFHRESP) is value 0, the same as the SQLCODE-OK value, so it conveniently doubles as the success test for both EIBRESP (EXEC CICS) and SQL-CODE (EXEC SQL). For the no-data case, test SQLCODE = +100 directly — there's no DFHSQL-CODE copybook yet.

Worked example: SQLD

runtime/cobol/sqld.cob (TRANSID SQLD) is the bundled demonstration. It reads a single customers_sql row by id and renders the name + SQLCA verdict on screen SQLD1. The expected schema is documented in the program header. The operator runs psql once to seed the table:

```sql
CREATE TABLE customers_sql (
    id   text PRIMARY KEY,
    name text NOT NULL
);
INSERT INTO customers_sql VALUES
    ('K001', 'Alice'),
    ('K002', 'Bob'),
    ('K003', 'Carol');
```

Then bricks □ sign on □ SQLD □ type K001 □ ENTER. The operator sees Alice in the NM cell with SQLCODE = 0 and SQLSTATE = 00000.

REXX equivalent

REXX programs use the same EXEC SQL shape as COBOL, with :name binding REXX variables. There's no DATA DIVISION analog – REXX is dynamically typed – so SQLCODE, SQLSTATE, SQLERRMC arrive as ordinary REXX variables the program reads after each statement:

```
ADDRESS CICS

CUSTID = 'K001'
```

```
EXEC SQL
    SELECT name INTO :NM
    FROM customers_sql
    WHERE id = :CUSTID
END-EXEC

IF SQLCODE = 0 THEN
    SAY 'Customer:' NM
ELSE IF SQLCODE = 100 THEN
    SAY 'Not found'
ELSE
    SAY 'SQL error' SQLCODE ':' SQLERRMC
```

Behind the scenes the REXX preprocessor (`rexx.PreprocessExecCICS`) rewrites the EXEC SQL block into a sentinel bare-string that `runCommand` recognises and routes to the same per-task SQL executor COBOL uses. The two language surfaces are parity-clean: the executor doesn't know or care which language drove it. Live sample: `runtime/rexx/sqlr.rexx`, TRANSID `SQLR`.

REXX stem-tail gotcha. Classic REXX resolves `STEM.NAME` by looking up the simple symbol `NAME` and using its value as the tail. If a program does `EXEC SQL ... INTO :NM` then writes `SCR.NM = NM`, the LHS resolves to `SCR.`<value-of-NM> = `SCR.Alice` — the value lands in the wrong slot. Workaround: pick a host-var name that is NOT also a stem tail. The bundled SQLR uses `CUSTNM` instead of `NM` for exactly this reason.

WHENEVER (declarative error handling)

Instead of testing `SQLCODE` after every statement, a program can declare a standing rule with `EXEC SQL WHENEVER`:

```
EXEC SQL WHENEVER SQLERROR  GO TO SQL-ERROR-EXIT END-EXEC.
EXEC SQL WHENEVER NOT FOUND GO TO NO-MORE-ROWS  END-EXEC.

EXEC SQL SELECT name INTO :NM FROM customers_sql
         WHERE id = :CUSTID END-EXEC.
*> no SQLCODE test here -- WHENEVER handles it
```

Three conditions, checked after every subsequent `EXEC SQL`:

Condition	Fires when
`SQLERROR`	`SQLCODE` is negative
`NOT FOUND`	`SQLCODE` = +100
`SQLWARNING`	`SQLWARN0` = `'W'` (bricks rarely sets this — PG errors rather than warns)

Each condition takes one of two actions:

- `CONTINUE` — ignore the condition, fall through (the default for any condition never declared).
- `GO TO label` / `GOTO label` — branch to the paragraph (COBOL) or label (REXX). The branch unwinds any enclosing `PERFORM` exactly as an explicit `GO TO` would.

A later `WHENEVER` for the same condition replaces the earlier one (e.g. declare `GO TO` for a block of statements, then `CONTINUE` to resume manual handling).

REXX uses the identical syntax; the branch is a `SIGNAL` to the named label:

```
EXEC SQL WHENEVER SQLERROR GOTO SQLERR END-EXEC
EXEC SQL SELECT name INTO :CUSTNM FROM customers_sql
         WHERE id = :CUSTID END-EXEC
SAY 'customer:' CUSTNM
EXIT
SQLERR:
SAY 'SQL failed, SQLCODE' SQLCODE
EXIT
```

Scoping caveat. bricks's `WHENEVER` is *execution-order* scoped: the directive set by the most recently **executed** `EXEC SQL WHENEVER` governs later statements. Real DB2 scopes *lexically* (by source position) because its precompiler inlines a `SQLCODE` test after every statement. For the dominant pattern — one `WHENEVER` near the top of the program — the two are identical. Programs that re-declare `WHENEVER` inside conditional branches see execution-order semantics; keep WHENEVER declarations at the top of a paragraph to avoid surprises.

Cursors (DECLARE / OPEN / FETCH / CLOSE)

Multi-row result sets use the four-verb cursor lifecycle. The executor parks one `*sql.Rows` per cursor per task; cursors are closed automatically on `COMMIT`, `ROLLBACK`, or task end so a program that forgets `CLOSE` doesn't pin Postgres MVCC.

```
EXEC SQL DECLARE C1 CURSOR FOR
    SELECT id, name FROM customers_sql ORDER BY id
END-EXEC.

EXEC SQL OPEN C1 END-EXEC.

PERFORM UNTIL SQLCODE = 100
    EXEC SQL FETCH C1 INTO :ID, :NM END-EXEC
    IF SQLCODE = 0 THEN
        DISPLAY 'row: ' ID ' ' NM
    END-IF
END-PERFORM.

EXEC SQL CLOSE C1 END-EXEC.
```

REXX uses the same four verbs (the preprocessor routes them all through the same executor):

```
EXEC SQL DECLARE C1 CURSOR FOR SELECT id, name FROM customers_sql END-EXEC
EXEC SQL OPEN C1 END-EXEC
DO FOREVER
    EXEC SQL FETCH C1 INTO :ID, :CUSTNM END-EXEC
    IF SQLCODE = 100 THEN LEAVE
    SAY 'row:' ID CUSTNM
```

```
END
EXEC SQL CLOSE C1 END-EXEC
```

End-of-data is `SQLCODE = +100`. A closed cursor stays in the registry, so `OPEN c1` can rewind it without re-`DECLARE`.

CONNECT TO 'name' (per-task database switch)

A program that needs to touch a second database during a task issues `EXEC SQL CONNECT TO 'name'`. The named database must exist in `databases.conf` (the CEDA DATABASE catalogue); an unknown name returns `SQLCODE = -1`. An in-flight PG transaction on the previous database is committed implicitly (matches DB2's behaviour) – programs that want to discard the previous work should `EXEC SQL ROLLBACK` before connecting.

```
EXEC SQL CONNECT TO 'orders' END-EXEC.
EXEC SQL INSERT INTO order_lines VALUES (:ID, :QTY) END-EXEC.
EXEC SQL COMMIT END-EXEC.
EXEC SQL CONNECT TO 'customers' END-EXEC.
EXEC SQL SELECT email INTO :EM FROM customers WHERE id = :CID END-EXEC.
```

Implicit binding still applies to the first statement of every task – the transaction's `Database` column in `transactions.conf` picks the starting pool. `CONNECT TO` is only needed when the program needs to move between databases inside a single task.

Null indicators (`:hostvar :indicator`)

DB2 programs use a second host var beside the value to signal NULL. Bricks supports the same pair syntax in both COBOL and REXX:

```
EXEC SQL
    SELECT name INTO :NM :NIND
    FROM customers_sql WHERE id = :CUSTID
END-EXEC.

EVALUATE NIND
    WHEN 0   DISPLAY 'Name: ' NM
    WHEN -1  DISPLAY 'Name is NULL'
END-EVALUATE.
```

Output side (SELECT INTO / FETCH INTO):

- Column was non-NULL indicator = `0`, value holds the result.
- Column was NULL indicator = `-1`, value is cleared to the empty string so a program that ignored its indicator can't read a stale value from a previous statement.

Input side (WHERE / VALUES / SET clauses):

```
MOVE -1 TO NIND.
EXEC SQL
    INSERT INTO customers_sql (id, name) VALUES (:K, :NM :NIND)
END-EXEC.
```

- Indicator = `-1` at bind time □ bricks passes NULL to PG, regardless of what `NM` currently holds.
- Indicator = `0` (or unset) □ the value var's frame contents bind normally.

Pair detection: a value/indicator pair is two `:name` tokens separated only by whitespace. `:NM, :NIND` (with a comma) means two independent host vars, not a pair. The `INDICATOR` keyword form (`:NM INDICATOR :NIND`) is not currently supported – use the juxtaposed form.

Column-value coercion

PG's wire format doesn't always match what COBOL or REXX want to read. Bricks normalises three cases before the value reaches the program's host variable:

- **`bool` columns** – PG returns `"t"` / `"f"` (or `"true"` / `"false"`). Bricks converts to `"1"` / `"0"` so COBOL PIC X(1) flags and REXX `IF VAR = 1` idioms work the way they do in DB2.
- **`NUMERIC(p, s)` columns** – PG drops trailing fractional zeros for computed expressions (`SELECT 1.5` returns `"1.5"` even though the result type has scale 2). Bricks pads the value to the column's declared scale, so a COBOL `PIC 9(3)V99` target receives `"1.50"` □ digits `"150"` □ stored as `001.50`, not `000.15`.
- **`CHAR(N)` columns** – PG returns space-padded values. Bricks trims trailing spaces so a REXX literal compare (`IF VAR = 'X'`) doesn't fail because the CHAR(5) value was `"X "`. VARCHAR is left alone (PG never pads it; its trailing spaces are real).

Other types (INT, BIGINT, TEXT, DATE, TIMESTAMP, JSON, …) flow through verbatim. COBOL's MOVE semantics handle integer sign and zero-pad automatically; REXX is dynamically typed so the string form works for both display and arithmetic.

Restrictions

Out of scope	Why
Stored procedure calls (`EXEC SQL CALL`)	Out of scope — porters use plain SELECT-from-function.
Multi-row INSERT VALUES (…), (…)	Out of scope; rewrite as one statement per row.
`CREATE` / `DROP DATABASE | USER`	CEDA DATABASE only (C / X actions, audit-logged).
Long-running cursors across `EXEC CICS` RETURN chains	Out of scope; each task gets a fresh PG connection.

Chapter 27. Restrictions and deferred features

Disallowed

- **Calculated GOTOs.** `GO TO DEPENDING ON` is rejected at parse time. The bricks runtime gives every task its own heap and stack with no static control-block aliasing; calculated GOTOs would require a per-program label-table the parser deliberately doesn't build.

Deferred Syntax Covrage

- Reference modification (`DATE-FIELD(1:4)` for substring access).
- Multi-dimensional `OCCURS`. One-level OCCURS is supported (see Chapter 22); the parser rejects an OCCURS item whose chain already contains another OCCURS ancestor.
- `SCREENHT`-based map family suffix (e.g. `CUST1L` on a mod-4 screen). REXX programs do this with a runtime `IF SCRH >= 43 THEN ...` fallback after a `MAPFAIL`; the COBOL twins always render the unsuffixed mod-2 maps for now.

Part 5. Sample programs

Chapter 28. Pre-installed sample transactions

COBOL

Transid	File	Notes
`HELC`	`runtime/cobol/hello.cob`	Hello-world; `SEND MAP('HELO1') FROM(SCR)`, `RETURN`. Smallest end-to-end demo.
`QAGC`	`runtime/cobol/qagc.cob`	COBOL twin of `QAGR` (REXX). Validates the QAGE1 birthdate, computes age in years and approximate days, sends QAGR1. Pseudo-conversational redisplay of QAGE1 on validation errors.
`GUST`	`runtime/cobol/gust.cob`	COBOL `CUST`. A=Add, Q=Query, U=Update, D=Delete, L=List, S=Search; the S action LINKs to GUSL with the search term in COMMAREA and renders the match count returned.
`GUSV`	`runtime/cobol/gusv.cob`	COBOL twin of `CUSV`. Validates the customer-number COMMAREA (LINK target).

Transid	File	Notes
GUSL	runtime/cobol/gusl.cob	COBOL twin of CUSL. Renders the customers file via STARTBR / READNEXT / ENDBR. Blank inbound COMMAREA = paginated all-records list (PF7/PF8). Non-blank inbound COMMAREA = filtered single-page browse: every record is FUNCTION POS-tested against the upper-cased filter, the first 15 matches populate ROW1..ROW15, and the total match count flows back through DFHCOMMAREA so the caller (GUST) can render a summary.
EXAM	runtime/cobol/exam.cob	Worked example of reading the operator's command-line arguments. Type EXAM 1 2 3 at the blank prompt.
ORDR	runtime/cobol/ordr.cob	Conversational import: reads runtime/tmp/orders.sample.txt via READQ TD, parses pipe-delimited rows, and WRITE FILE('ORDERS') keyed on customer-id. Tolerates duplicates (EIBRESP = RESP-DUPREC). Summary screen shows counts. See worked example E.
TIMC	runtime/cobol/timc.cob	Timed-reminder demo: exercises CONVERSE, START (with INTERVAL + FROM), and RETRIEVE end-to-end. Shares tim1.map + tim2.map with TIMR.

Transid	File	Notes
`SQLD`	`runtime/cobol/sqld.cob`	Embedded-SQL demo: `EXEC SQL SELECT name INTO :NM FROM customers_sql WHERE id = :CUSTID`. Renders the row + SQLCODE / SQLSTATE / SQLERRMC. Requires Postgres seeded with the schema shown in Chapter 26.
`SQLR`	`runtime/rexx/sqlr.rexx`	REXX twin of `SQLD`. Shares the `customers_sql` table; demonstrates how REXX accesses the same EXEC SQL surface (SQLCODE / SQLSTATE / SQLERRMC arrive as plain REXX variables).

All five non-trivial COBOL samples (`QAGC`, `GUST`, `GUSL`, `ORDR`, `EXAM`) `COPY DFHAID` and/or `COPY DFHRESP` instead of hard-coding hex AID bytes or numeric `EIBRESP` codes. Skim any of them as living examples of the named-constant idiom from Chapter 25.

REXX (selected)

Transid	File	Notes
`HELO`	`runtime/rexx/hello.rexx`	Hello-world with system-info pane on mod 4.
`CUST`	`runtime/rexx/cust.rexx`	Full customer maintenance — Add / Query / Update / Delete / List / Search; LINKs to `CUSV` for validation.
`CUSV`	`runtime/rexx/cusv.rexx`	Validation sub-program; called by `CUST` via `EXEC CICS LINK`.
`CUSL`	`runtime/rexx/cusl.rexx`	Paginated customer list using `STARTBR / READNEXT / ENDBR`; adapts paging to mod 2 vs mod 4.
`QAGE` / `QAGR`	`runtime/rexx/qage.rexx` / `qagr.rexx`	Pseudo-conversational chain; `QAGE` prompts for a birthdate and chains to `QAGR` to render the result.

Transid	File	Notes
PROD / CONS	runtime/rexx/prod.rexx / cons.rexx	TS queue producer / consumer pair. Conversational; PF3 to exit.
GETC	runtime/rexx/getc.rexx	RECEIVE of command-line + READ FILE + SEND TEXT (no map).
TIMR	runtime/rexx/timr.rexx	REXX twin of TIMC: START schedules a reminder; RETRIEVE discriminates cold vs scheduled entry. Shares tim1.map + tim2.map with TIMC.

Run any TRANSID by typing it at the blank prompt after CSSN sign-on. Refer to runtime/transactions.conf for the full list and ACL configuration.

Built-in transactions (no entry in `transactions.conf`)

The bricks core dispatches a handful of TRANSIDs directly, without consulting transactions.conf. They take precedence over a same-name entry in the table.

Transid	Purpose	Notes
CSSN	Sign-on	Default authentication flow. Configurable via secure_login_transacton in bricks.cnf.
CSSF	Sign-off	CSSF LOGOFF clears the session's identity; bare CSSF is a no-op.
CEMT	Master-operator	INQUIRE / MONITOR / PERFORM trees; CONTROLBLOCKS sub-tree and PERFORM gated on the admin group.
CEDA	Resource definitions	TRANSACTION / PROGRAM / USER screens; admin-only.

Transid	Purpose	Notes
`ISPF`	Source editor	Browse and edit the REXX, COBOL, and BMS-map source trees. Gated on the `dev` group. **Operator manual:** `ISPF_editor.md` — covers every PF key, every command-line word, every line-prefix command (D / I / C / M / R / U / L /) / (/ X / O / A / B plus the doubled block forms), the file browser, the warn-then-save flow, multi-file editing, and edit locks.

Chapter 29. Worked examples

A. Producer / consumer over a TS queue

`PROD` writes one item per ENTER from an interactive map; `CONS` reads with the implicit per-task cursor. PF4 in `CONS` deletes the queue; PF5 rewinds the cursor by chaining back to itself (`RETURN TRANSID('CONS')`), so the dispatcher's task-end hook clears the cursor.

```
/* PROD: write one item per ENTER */
ADDRESS CICS
DO FOREVER
  EXEC CICS SEND    MAP('PROD1') FROM(SCR.) ERASE END-EXEC
  EXEC CICS RECEIVE MAP('PROD1')                  END-EXEC
  IF C2X(EIBAID) = 'F3' THEN EXEC CICS RETURN END-EXEC
  EXEC CICS WRITEQ TS QUEUE(MAP.QNAME) FROM(MAP.PAYLOAD) END-EXEC
END
```

```
/* CONS: cursor-advancing read */
ADDRESS CICS
DO FOREVER
  EXEC CICS SEND    MAP('CONS1') FROM(SCR.) ERASE END-EXEC
  EXEC CICS RECEIVE MAP('CONS1')                  END-EXEC
  IF C2X(EIBAID) = 'F3' THEN EXEC CICS RETURN END-EXEC
  EXEC CICS READQ TS QUEUE(QNM) INTO(REC) ITEM(GOTI) END-EXEC
  IF EIBRESP = 26 THEN /* ITEMERR - end of queue */ NOP
END
```

B. SEND TEXT + RECEIVE — no BMS map

`GETC` takes a customer number from the operator's command line (`GETC 100`), reads the `customers` KSDS, and paints the record as free-form text. Because **3270 has no LF**, the body is built as a flat row-major buffer — each logical line padded to the 80-column screen width with `LEFT(s,80)`.

```
/* GETC: lookup-and-display, no map */
ADDRESS CICS
EXEC CICS RECEIVE INTO(BUF) END-EXEC          /* "GETC 100" */
PARSE VAR BUF TID CKEY .
EXEC CICS READ FILE('customers') INTO(REC) RIDFLD(CKEY) END-EXEC
PARSE VAR REC NM '|' AD '|' CY '|' PH
TXT = LEFT('Customer #' || CKEY, 80)          /* row 0 */
TXT = TXT || LEFT('', 80)                     /* row 1 (blank) */
TXT = TXT || LEFT('Name:    ' || NM, 80)      /* row 2 */
TXT = TXT || LEFT('Address: ' || AD, 80)      /* row 3 */
EXEC CICS SEND TEXT FROM(TXT) ERASE END-EXEC
EXEC CICS RETURN END-EXEC
```

The COBOL companion `runtime/cobol/exam.cob` shows the same `RECEIVE INTO` pattern with `UNSTRING ... DELIMITED BY ' '` doing the tokenisation:

```
EXEC CICS RECEIVE INTO(WS-INPUT) LENGTH(WS-LEN) END-EXEC
UNSTRING WS-INPUT DELIMITED BY ' '
   INTO WS-TID WS-A WS-B WS-C
END-UNSTRING.
```

C. Browse with GENERIC prefix

A paginated customer list filtered by a 3-character key prefix:

```
EXEC CICS STARTBR FILE('CUSTOMERS')
                 RIDFLD('NY-')
                 GENERIC KEYLENGTH(3)
END-EXEC
DO FOREVER
  EXEC CICS READNEXT FILE('CUSTOMERS') INTO(REC) RIDFLD(K) END-EXEC
  IF EIBRESP = 20 THEN LEAVE          /* ENDFILE */
  CALL ADD_ROW K, REC
END
EXEC CICS ENDBR FILE('CUSTOMERS') END-EXEC
```

D. Synchronous LINK with COMMAREA

CUST LINKs to CUSV to validate a customer number:

```
SAV = CKEY                                      /* in: number to validate */
EXEC CICS LINK PROGRAM('CUSV') COMMAREA(SAV) END-EXEC
PARSE VAR SAV STATUS '|' MSG                    /* out: status, message  */
IF STATUS <> 'OK' THEN ...
```

CUSV reads `DFHCOMMAREA`, runs its check, and writes the result back into `DFHCOMMAREA` before `RETURN`. The dispatcher hands the final value back to the caller's `SAV`.

E. Sequential import via READQ TD + WRITE FILE

The ORDR transaction (runtime/cobol/ordr.cob) demonstrates the "text file □ VSAM" pattern that the tmp_dir sandbox is built for. The sample data ships in runtime/tmp/orders.sample.txt:

```
C0000001|WIDGET-A|10|19.95
C0000002|WIDGET-B|2|249.00
...
```

The loop body is just two EXEC CICS calls and an UNSTRING:

```
IMPORT-ONE.
    MOVE SPACES TO REC.
    EXEC CICS READQ TD QUEUE('orders.sample.txt') INTO(REC) END-EXEC.
    IF EIBRESP = 12 THEN
        MOVE 'Y' TO DONE-FLAG       *> QZERO -- end of file
    END-IF.
    IF EIBRESP = 0 THEN
        COMPUTE N-READ = N-READ + 1
        MOVE SPACES TO CUST-ID PRODUCT QTY PRICE
        UNSTRING REC DELIMITED BY '|'
            INTO CUST-ID PRODUCT QTY PRICE
        END-UNSTRING
        PERFORM WRITE-ORDER
    END-IF.

WRITE-ORDER.
    MOVE SPACES TO OREC.
    STRING PRODUCT DELIMITED BY SIZE
           '|' DELIMITED BY SIZE
           QTY DELIMITED BY SIZE
           '|' DELIMITED BY SIZE
           PRICE DELIMITED BY SIZE
        INTO OREC
    END-STRING.
    EXEC CICS WRITE FILE('ORDERS') FROM(OREC) RIDFLD(CUST-ID) END-EXEC.
    IF EIBRESP = 0  THEN COMPUTE N-WRITE = N-WRITE + 1 END-IF.
    IF EIBRESP = 14 THEN COMPUTE N-DUP   = N-DUP   + 1 END-IF.
```

Notes:

- The READQ loop tracks EOF with EIBRESP = 12 (QZERO) — the CICS-canonical "queue empty / no more records" signal. The handle is closed automatically at task end.
- EIBRESP = 14 (DUPREC) is treated as a counted no-op, not an error: WRITE FILE against an existing key fails atomically without overwriting, so a re-run of ORDR on the same file is idempotent on the customer-id set.
- ORDR ships public in runtime/transactions.conf so the sample runs out of the box. In a production deployment that imports real data, restrict it to a privileged group (ORDR:cobol:ordr.cob:admin) so a casual operator can't replay the import.
- A REXX equivalent reads the same file via LINEIN and writes via EXEC CICS WRITE. Be-

cause both languages share the `tmp_dir` backend, the sample file works untouched from either side.

Appendix A. Adapting to terminal size (mod 2 vs mod 4)

A 3270 connection negotiates one of several screen models — typically mod 2 (24 - 80) or mod 4 (43 - 80). Bricks captures the size from the telnet/3270 handshake into `session.TCB.Rows/Cols`, and exposes it to programs via `EXEC CICS ASSIGN`:

```
EXEC CICS ASSIGN SCREENHT(SCRH) SCREENWD(SCRW)
                 ALTSCRNHT(AH)  ALTSCRNWD(AW)  END-EXEC
```

`SEND MAP` always passes the negotiated `DevInfo` through to `go3270.ScreenOpts.AltScreen`, so the underlying datastream uses Erase/Write Alternate (`0x7e`) and the terminal clears its full buffer — but a 24-row map painted on a 43-row screen leaves rows 24-42 blank. To use the extra real estate the program has to dispatch to a sized map variant.

Convention

Author one map per model. The mod-2 map keeps its bare name; bigger models add a single-letter suffix:

Model	Suffix	Example map names
mod 2	(none)	`HELO1`, `CUST1`, `CUST2`, `CUSTL`
mod 3	`M`	`HELO1M`, `CUST1M`, …
mod 4	`L`	`HELO1L`, `CUST1L`, `CUST2L`, `CUSTLL`
mod 5	`W`	`HELO1W`, …

The REXX program builds the suffixed name once and dispatches:

```
EXEC CICS ASSIGN SCREENHT(SCRH) END-EXEC
SUFFIX = ''
IF SCRH >= 43 THEN SUFFIX = 'L'
ELSE IF SCRH >= 32 THEN SUFFIX = 'M'

EXEC CICS SEND MAP('HELO1' || SUFFIX) FROM(SCR.) ERASE END-EXEC
IF EIBRESP = 36 THEN DO            /* MAPFAIL — sized variant missing */
  EXEC CICS SEND MAP('HELO1') FROM(SCR.) ERASE END-EXEC
END
```

Three properties make this work without any DSL changes:

1. **Same field names across the family.** `helo1.map` and `helo1l.map` both declare `INFO-LINE`, `GREETING`, `FOOTER`. The same `SCR.` stem feeds either one. Bonus tails (e.g. `INFO1` / `INFO2` / `ACT1` …) are silently ignored on the smaller map (the renderer only writes values for fields that the map declares).

2. **MAPFAIL fallback.** If the suffixed map isn't on disk, the `SEND` returns `EIBRESP = 36` and the program retries with the bare name — so an operator who deletes `helo1l.map` doesn't break mod-4 connections; they just see the 24-80 layout.
3. **Paging arithmetic adapts at runtime.** `cusl.rexx` reads `SCREENHT` and uses `ROWS_PER_PAGE = 35` on mod 4 vs `15` on mod 2, then picks `CUSTLL` vs `CUSTL` accordingly.

Bricks ships sized variants for every map the demo transactions use:

Mod-2 map (24-80)	Mod-4 sibling (43-80)
`runtime/map/helo1.map`	`runtime/map/helo1l.map` — adds system-information + recent-activity panes
`runtime/map/cust1.map` (menu)	`runtime/map/cust1l.map` — adds recent-activity history
`runtime/map/cust2.map` (detail)	`runtime/map/cust2l.map` — adds an audit-log pane
`runtime/map/custl.map` (15-row list)	`runtime/map/custll.map` (35-row list)

To see the difference: connect with `c3270 -model 2 localhost 2300` vs. `c3270 -model 4 localhost 2300`, sign on, and run `HELO` or `CUST`. The mod-4 view fills the bottom three-quarters of the screen with extra panels.

Appendix B. Pitfalls and idioms

REXX compound-symbol pitfall

`STEM.tail` with `tail` an *unset* symbol resolves to `STEM.<TAIL>` (a literal tail). With `tail` a *set* symbol it resolves to `STEM.<value-of-tail>` — so reusing a map field name (`OUT.BIRTH = …` when `BIRTH` is also a local variable) silently writes the wrong tail.

The convention used by `runtime/rexx/cust.rexx` is to give locals distinct names from map fields (`AKT` vs `ACTION`, `CKEY` vs `CUSTNO`, `BSTR` / `NDAYS` vs `BIRTH` / `DAYS`).

COBOL data names across groups

A child name reused across groups is allowed; bare references that match more than one declaration must be qualified with `OF` / `IN` (`MOVE x OF DET TO y OF SCR`). Bare names that are unique across the program continue to work without qualification. Sibling duplicates within a single group are still rejected at parse time because no qualifier can disambiguate them.

3270 has no LF…obviously.

`SEND TEXT` lays its body out as a flat row-major buffer, every `cols` bytes wrapping to the next row. Programs that expect newline semantics must pad each logical line to the column width (`LEFT(s,80)` in REXX, `PIC X(80)` group children in COBOL) and concatenate.

Kinda obvious, but just making sure it's clear to folks who are new to 3270.

Reset stems before paginated reads

A REXX `STEM.` accumulator that's reused across pages will leak values from the previous page if not dropped. Use `DROP RECS.` (with the trailing dot) at the top of each pagination loop.

EIBAID comparison

In REXX, compare `C2X(EIBAID)` to a hex string: `IF C2X(EIBAID) = 'F3' THEN ...` (PF3). In COBOL, compare `EIBAID` directly to a hex literal: `IF EIBAID = X'F3' ...`.